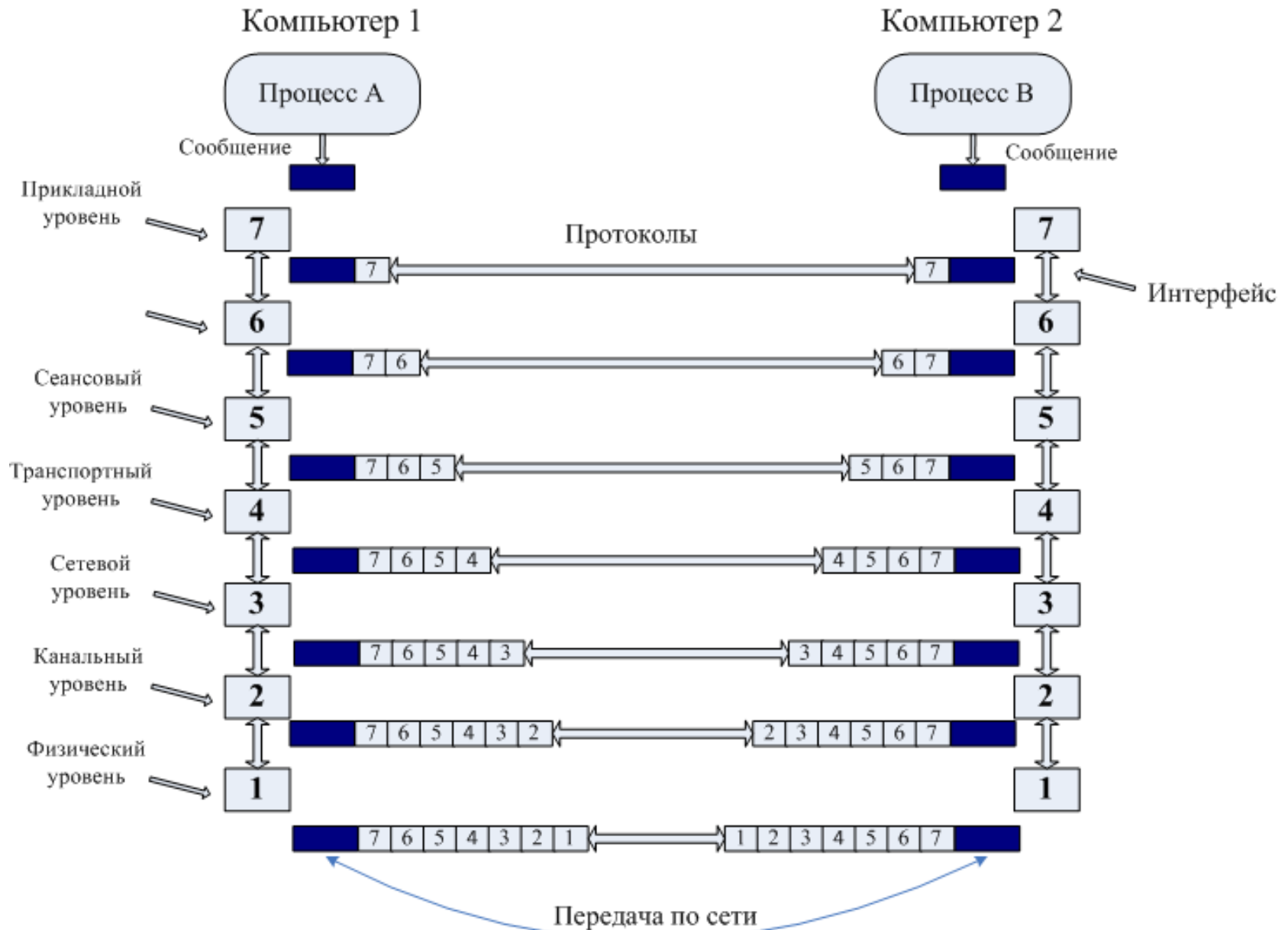


Сетевое программирование

Профессор Кафедры вычислительных систем
доктор технических наук
доцент

Павский Кирилл Валерьевич

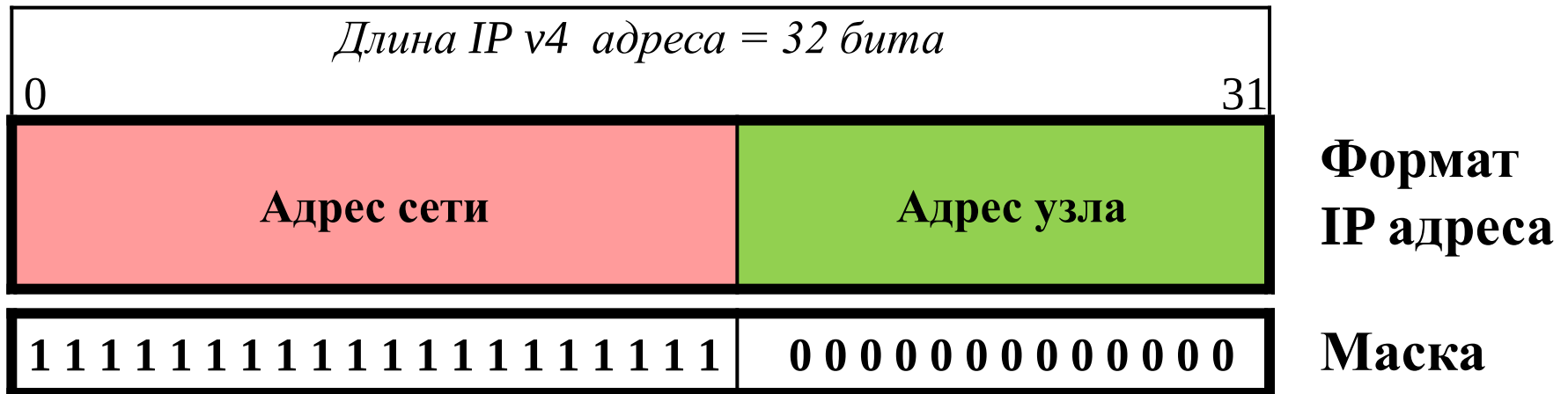
Модель OSI



Модель TCP/IP

Модель OSI	TCP/IP	Стек OSI
Прикладной	Telnet, FTP, POP3, SMTP, IMAP4, ICQ, SNMP, WWW	X.400, X.500, FTAM
Представительный		Представительный протокол OSI
Сеансовый		Сеансовый протокол OSI
Транспортный	TCP, UDP, SCTP	Транспортный протокол OSI
Сетевой	IP, RIP, OSPF, ICMP	ES-ES, IS-IS
Канальный	802.3 (Ethernet), 802.5 (Token Ring), Fast Ethernet, X.25	
Физический	Витая пара, оптоволокно, радиоволны	

IP v.4адресация. Маски.



Маска – это число, которое используется в паре с IP-адресом и в битах, которые обозначают адрес сети, содержит единицы, а в битах номера хоста – нули.

Классы IP-сетей

Класс	1 байт			2 байт		3 байт		4 байт	
A	0	Сеть			Хост				
B	1	0	Сеть				Хост		
C	1	1	0	Сеть					Хост
D	1	1	1	0	Групповой адрес				
E	1	1	1	1	0	Зарезервировано			

IP v.4адресация. Маски

Адресное пространство классов IP-сетей

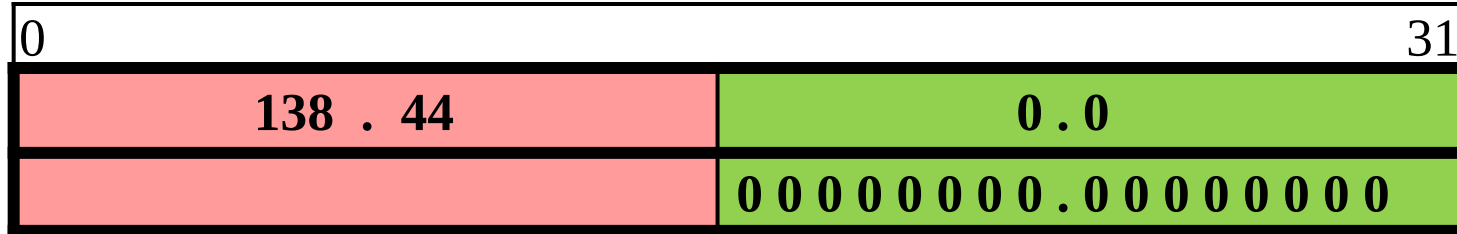
Класс	Наименьший номер сети	Наибольший номер сети	Маска	Комментарии
A	1.0.0.0	126.0.0.0	255.0.0.0	До 2 ²⁴ узлов
	10.0.0.0	10.0.0.0	255.0.0.0	Внутренние закрытые сети
B	128.0.0.0	191.255.0.0	255.255.0.0	До 2 ¹⁶ узлов
	172.16.0.0	172.31.0.0	255.255.0.0	Внутренние закрытые сети
C	192.0.0.0	223.255.255.0	255.255.255.0	До 2 ⁸ узлов
	192.168.0.0	192.168.255.0	255.255.255.0	Внутренние закрытые сети
D	224.0.0.0	239.255.255.255		Групповой адрес
	224.0.0.1	224.0.0.1		Группа хостов в данной сети
E	240.0.0.0	247.255.255.255		Зарезервирован

Маски. Пример организации сети

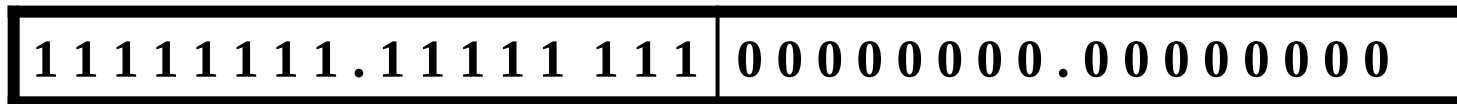
Задача:

Администратор получил в свое распоряжение адрес класса В: **138.44.0.0**.

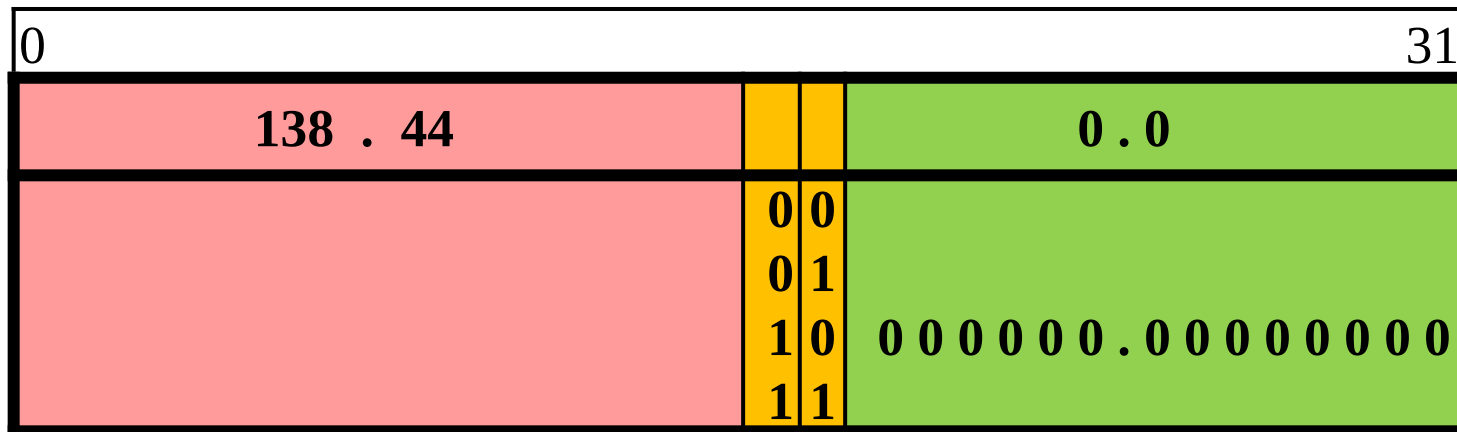
Требуется, чтобы сеть была разделена на три отдельных подсети.



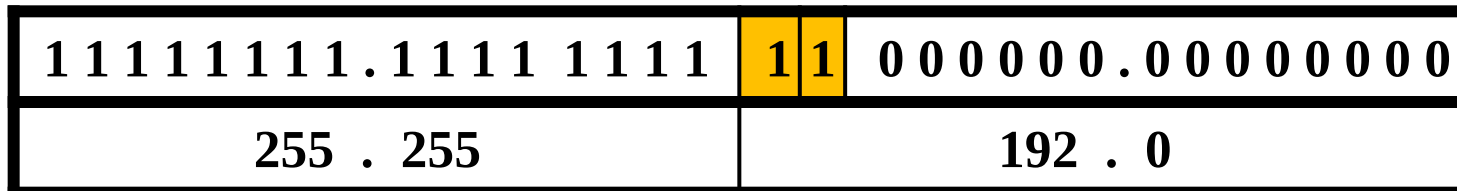
Одна сеть



Маска



*Четыре
подсети*



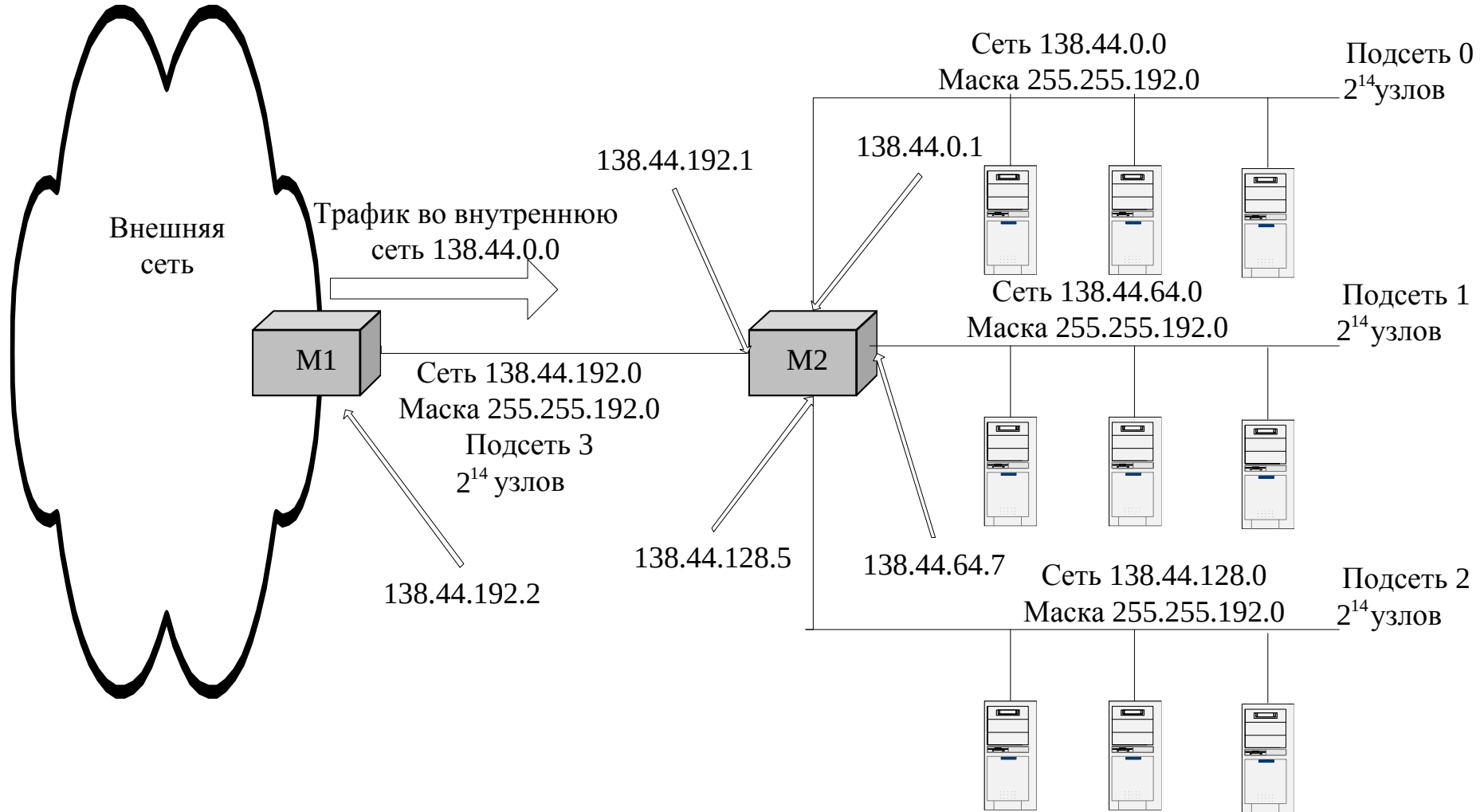
Маска

В качестве маски было выбрано значение **255.255.192.0**

Маски. Пример организации сети

Номер сети	Маска	Адрес следующего маршрутизатора	Адрес порта	Расстояние
138.44.0.0	255.255.192.0	138.44.0.1	138.44.0.1	Подключена
138.44.64.0	255.255.192.0	138.44.64.7	138.44.64.7	Подключена
138.44.128.0	255.255.192.0	138.44.128.5	138.44.128.5	Подключена
138.44.192.0	255.255.192.0	138.44.192.1	138.44.192.1	Подключена
0.0.0.0	0.0.0.0	138.44.192.2	138.44.192.1	—

Пример организации сети



Структура IP-пакета

Строка	0	34	78	15	16	31		
0	Version	IHL	TOS					Total Length
			PR	D	T	R		
1	Identification					Flags		Fragment Offset
							D	
2	TTL		Protocol			Header Checksum		
3	Source Address							
4	Destination Address							
5-14	Options					Padding		

Формат заголовка IP-пакета

Фрагментация IP-пакетов

Хотя максимальная длина датаграммы IP – 64 КБ, большинство сетей устанавливают максимальный предел длины пакета

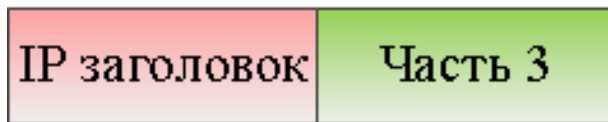
Если размер IP пакета больше MTU ,
Что делать?



IP протокол предоставляет возможность приспособиться к различным MTU, разрешая маршрутизаторам фрагментировать IP пакеты по мере необходимости.



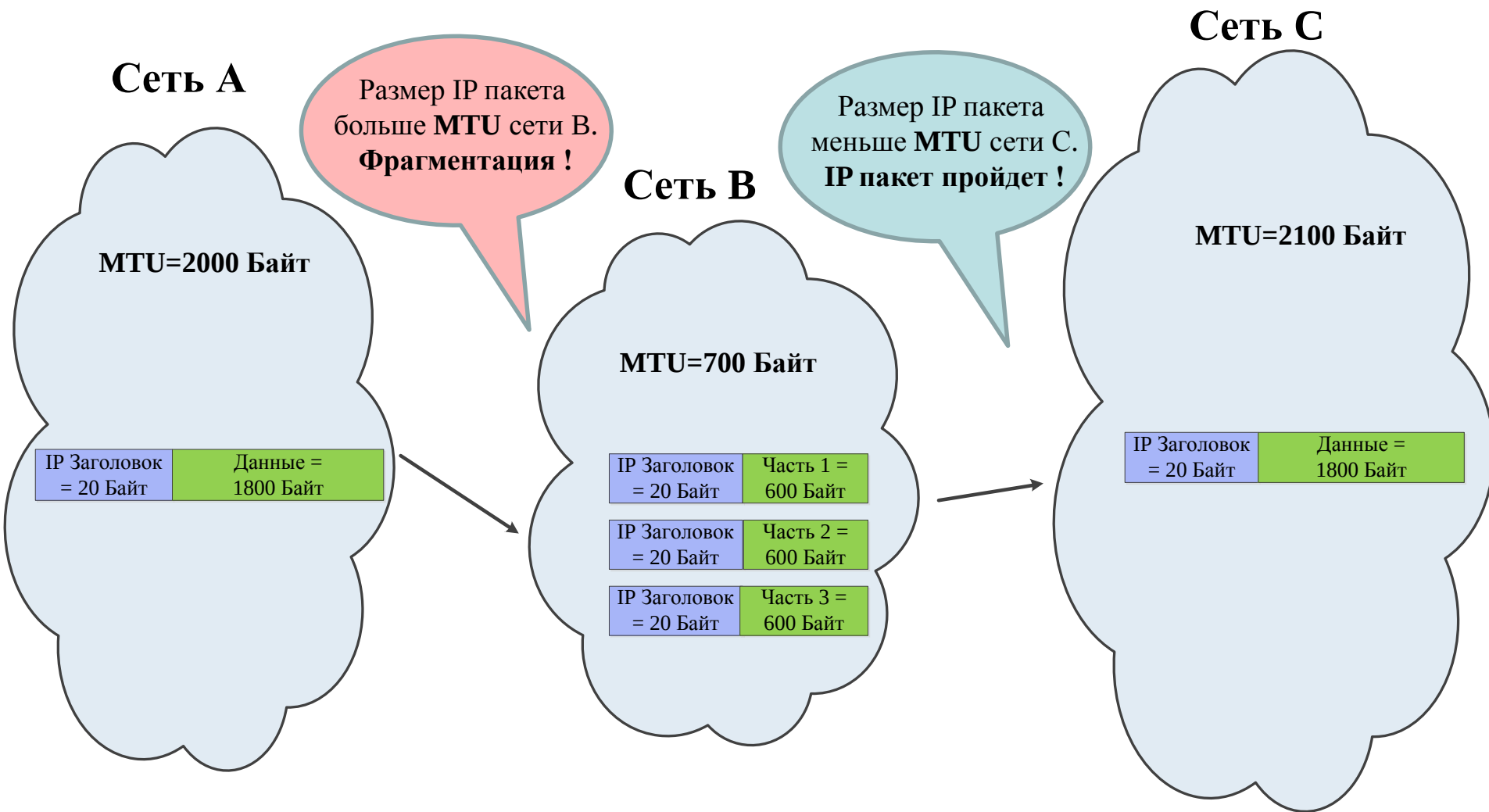
Процедура фрагментации заключается в разбиении IP пакета на несколько фрагментов, которые позже будут собраны в исходный пакет.



Теперь
каждый фрагмент исходного
IP пакета меньше MTU .
Пакет пройдет!
Однако, ...

Фрагментация IP-пакета
увеличивает нагрузку на центральный
процессор и снижает скорость передачи
полезных данных этого пакета, поэтому её
стараяются избегать !!!

Фрагментация IP-пакетов



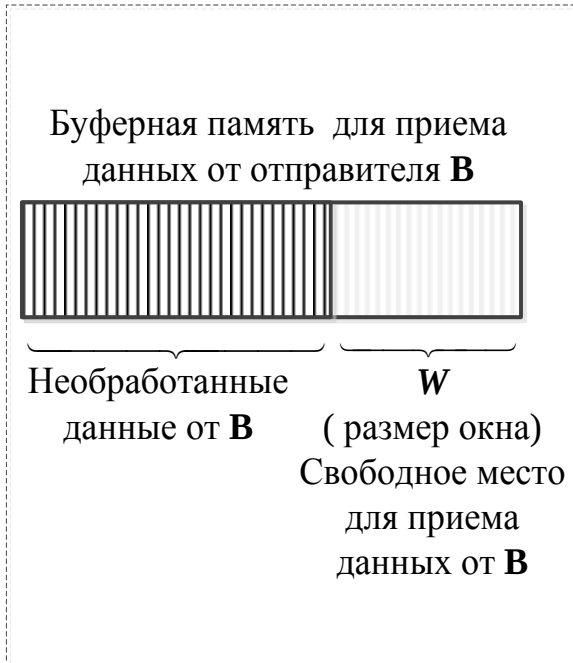
Структура TCP-пакета

Строка	0	7	8	15	16	31		
0	Source Port				Destination Port			
1	Sequence Number							
2	Acknowledgement Number							
3	Data Offset	Reserved	URG	ACK	PSH	SYN	FIN	Window
4	Checksum				Urgent Pointer			
5-14	Options					Padding		

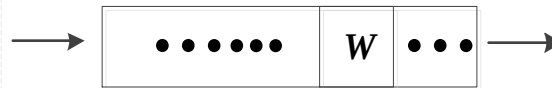
Формат заголовка TCP-пакета

Процедура «окно» протокола ТСР

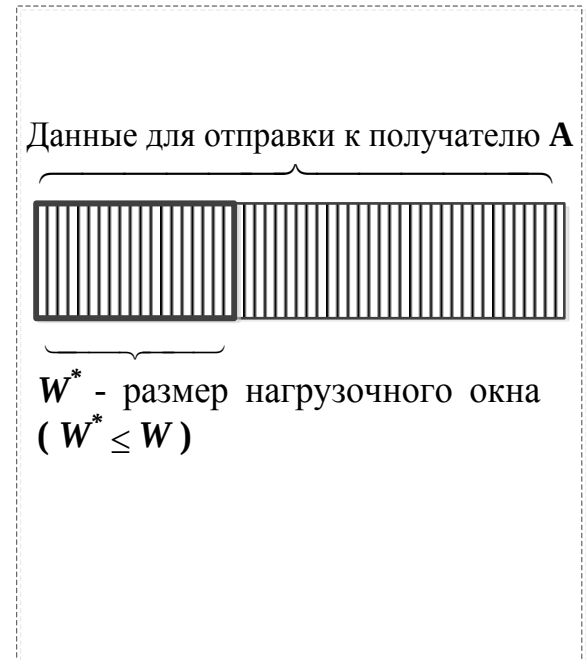
Получатель А



Заголовок ТСР

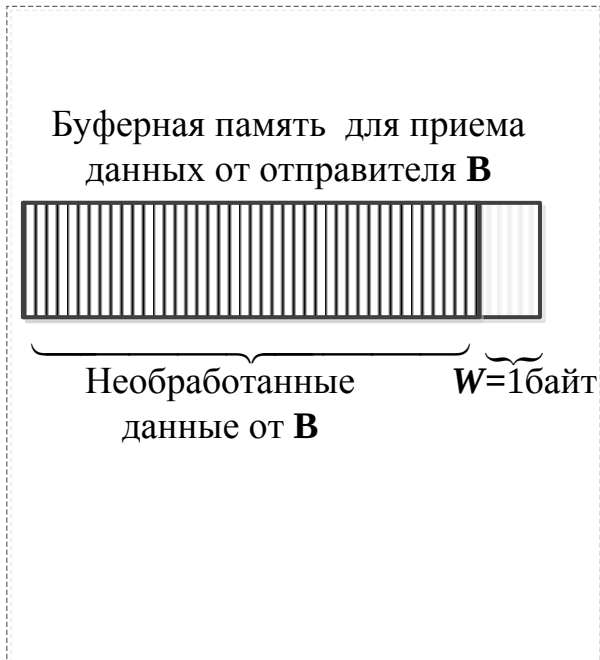


Отправитель В

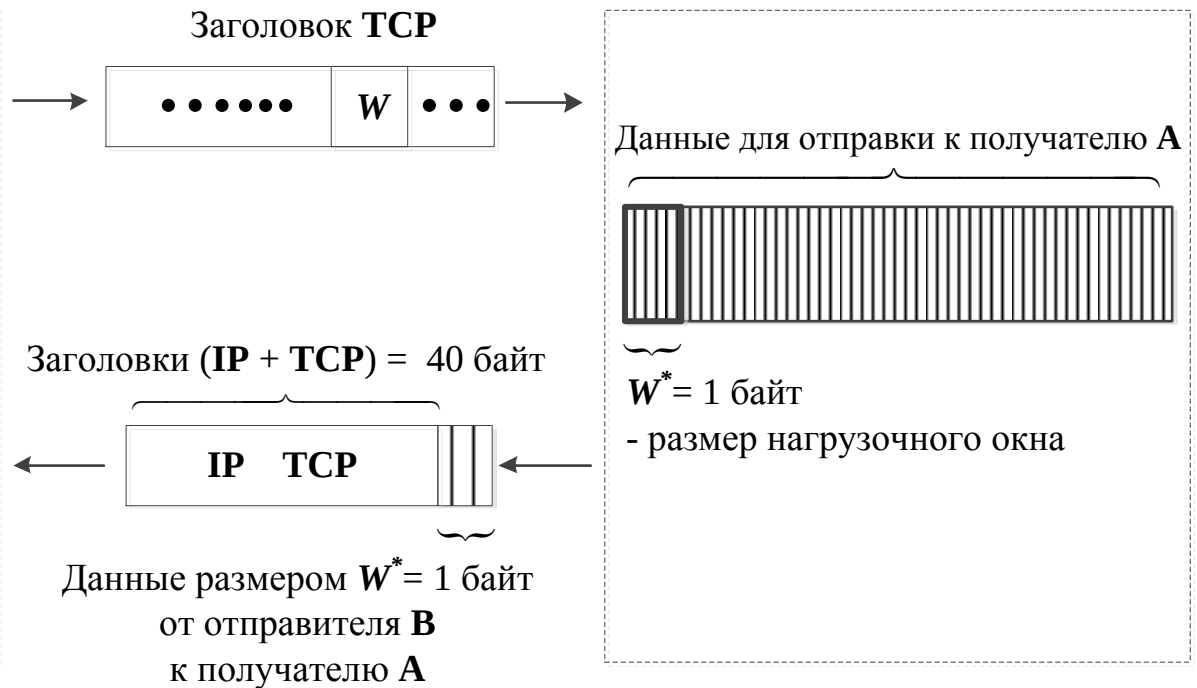


Феномен синдрома «бестолкового окна»

Получатель А



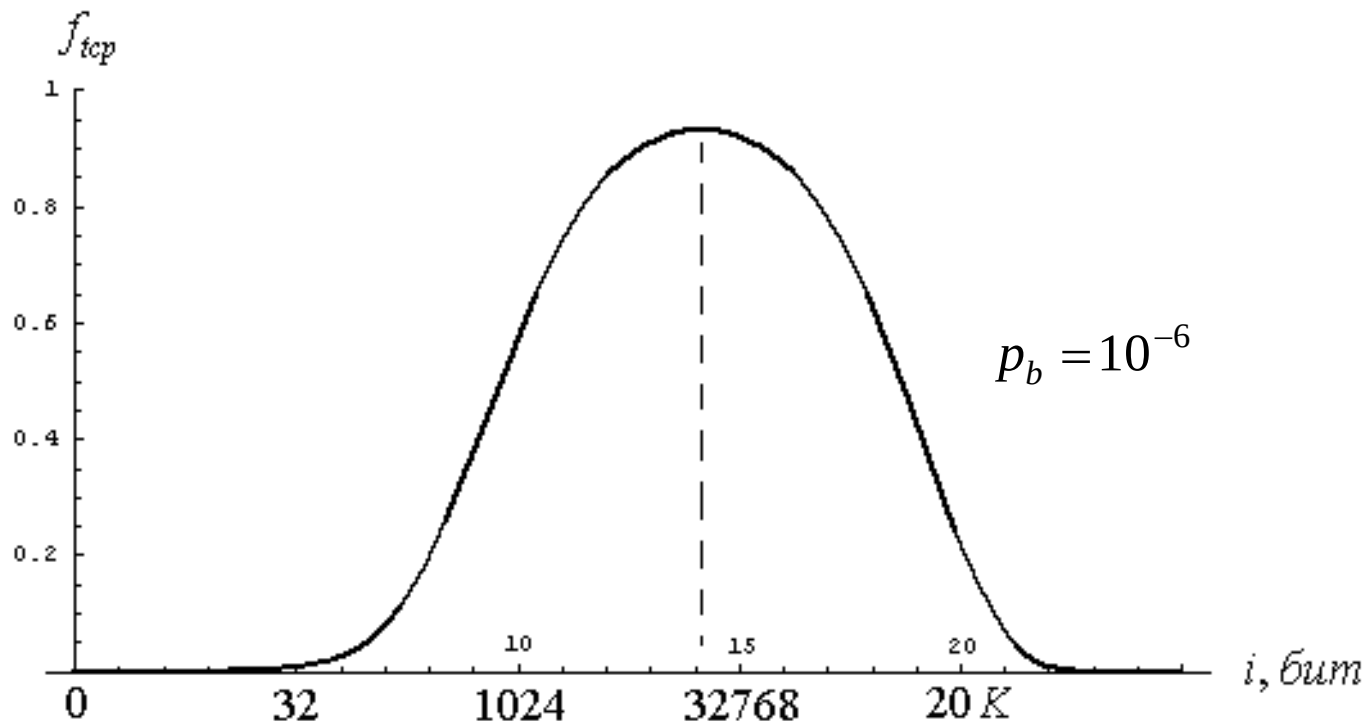
Отправитель В



Эффективность передачи информации

$$f_{tcp}(i) = \frac{(1 - p_b)^{i+s}}{1 + \frac{s}{i+s} (1 - (1 - p_b)^{i+s})} \cdot \left(\frac{i}{i+s} \right)$$

где S - размер заголовка пакета в битах.



Эффективность передачи данных в зависимости от размера информационной части
Заголовок пакета занимает 320 **бит**.

Алгоритм Дейкстры

Пример:

	1	2	3	4	5	6
1	0	3	6	2	<i>inf</i>	<i>inf</i>
2	3	0	4	3	<i>inf</i>	<i>inf</i>
3	6	4	0	4	2	2
4	2	3	4	0	1	<i>inf</i>
5	<i>inf</i>	<i>inf</i>	2	1	0	3
6	<i>inf</i>	<i>inf</i>	2	<i>inf</i>	3	0

Матрица графа расстояний

Необходимо найти граф минимальных расстояния от 1 вершины до всех остальных вершин.

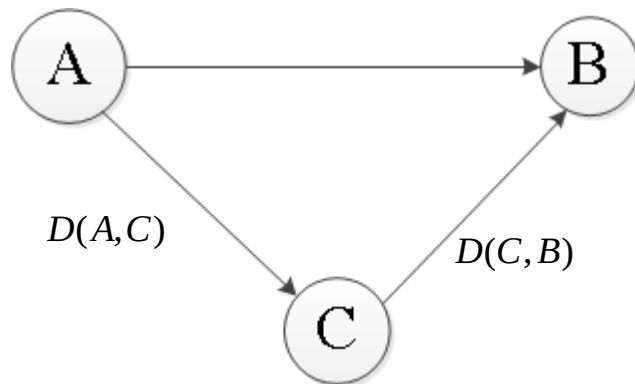
Алгоритм Дейкстры

	1	2	3	4	5	6
1	0	3	6	2	inf	inf
2	3	0	4	3	inf	inf
3	6	4	0	4	2	2
4	2	3	4	0	1	inf
5	inf	inf	2	1	0	3
6	inf	inf	2	inf	3	0

Матрица графа расстояний

№ множество	2	3	4	5	6
1	3 ₁	6 ₁	2 ₁	inf	inf
1,4					

Расчет кратчайшего расстояния от 1-й вершины



if $D(A, B) > D(A, C) + D(C, B)$
 then $D(A, B) = D(A, C) + D(C, B)$

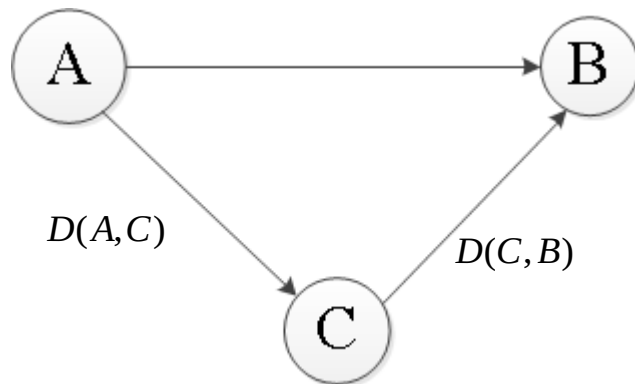
Алгоритм Дейкстры

	1	2	3	4	5	6
1	0	3	6	2	inf	inf
2	3	0	4	3	inf	inf
3	6	4	0	4	2	2
4	2	3	4	0	1	inf
5	inf	inf	2	1	0	3
6	inf	inf	2	inf	3	0

Матрица графа расстояний

№ множество	2	3	4	5	6
1	3 ₁	6 ₁	2 ₁	inf	inf
1,4	3 ₁	6 ₁	2 ₁	3 ₄	inf
1,4,2	3 ₁	6 ₁	2 ₁	3 ₄	inf
1, 4, 2, 5	3 ₁	5 ₅	2 ₁	3 ₄	6 ₅
1, 4, 2, 5, 3	3 ₁	5 ₅	2 ₁	3 ₄	6 ₅

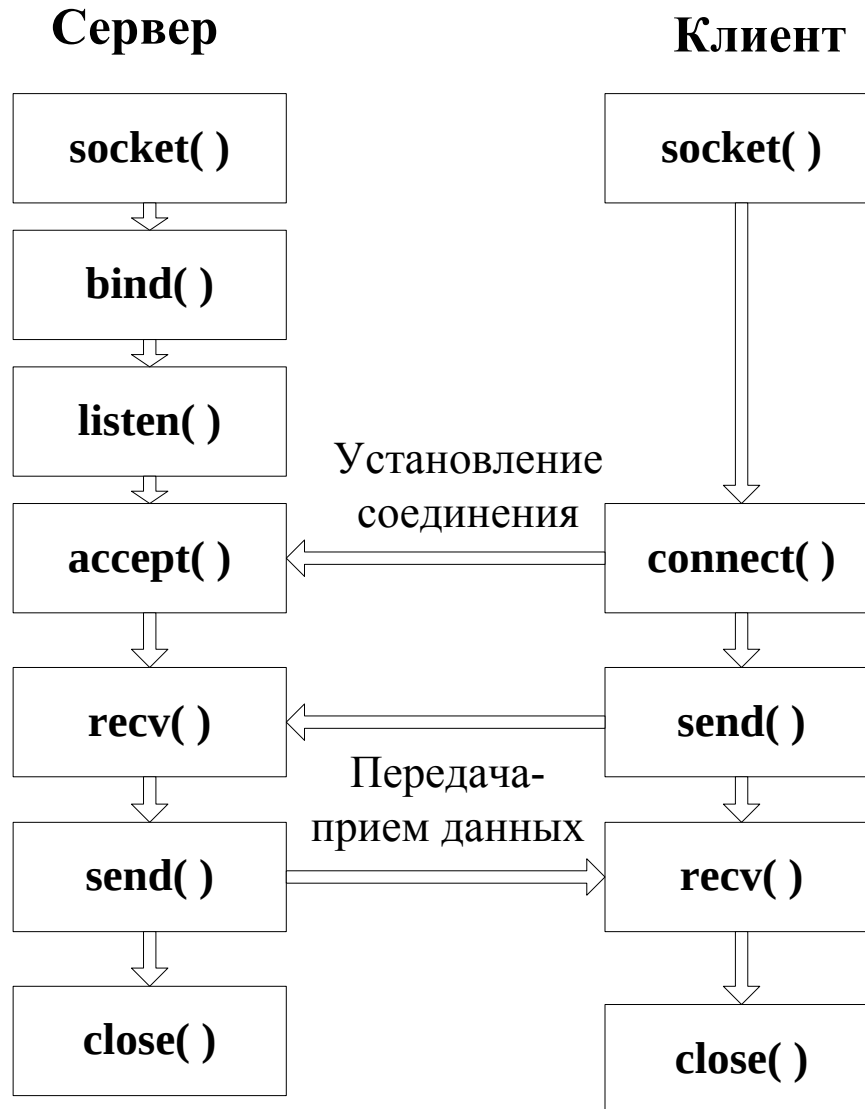
Расчет кратчайшего расстояния от 1-й вершины



if $D(A, B) > D(A, C) + D(C, B)$
 then $D(A, B) = D(A, C) + D(C, B)$

API Socket

Программа типа клиент-сервер для ТСР



API Socket

```
#include <sys/types.h>  
#include <sys/socket.h>
```

int socket (int domain, int type, int protocol);

Аргумент *domain* определяет правила именования гнезда и формат адреса, используемые в протоколе. Широко применяются такие домены, как ***AF_UNIX*** (домен UNIX) и ***AF_INET*** (Internet-домен).

Аргумент *type* задает тип гнезда.

Аргумент *protocol* указывает конкретный протокол, который следует использовать с данным гнездом. Как правило, оно устанавливается в 0, и операционная система сама выбирает для указанного типа гнезда соответствующий протокол. Например, для типа гнезда ***SOCK_STREAM*** третьему аргументу по умолчанию выбирается значение TCP, а для ***SOCK_DGRAM*** устанавливается UDP.

В случае успешного выполнения рассматриваемая функция возвращает целочисленный дескриптор гнезда, а в случае неудачи возвращает -1.

int close (int sid); Закрытие сокета *sid* .

API Socket

```
int bind ( int sid, struct sockaddr* addr_p, int len );
```

Эта функция присваивает гнезду имя. Гнездо обозначается аргументом *sid*, значение которого, возвращенное функцией *socket*, представляет собой дескриптор гнезда. Аргумент *addr_p* указывает на структуру, содержащую имя, которое должно быть присвоено гнезду. Аргумент *len* задает размер структуры, на которую указывает аргумент *addr_p*.

```
struct sockaddr_in {  
    short    sin_family;  
    u_short sin_port;  
    struct in_addr sin_addr;  
};
```

Здесь полю *sin_family* следует присвоить значение *AF_INET*.

Поле *sin_port* — это номер порта, а поле *sin_addr* — имя хост-машины, для которой создается гнездо. Структура *sockaddr_in* определяется в заголовке `<netinet/in.h>`.

Замечание. Там, где вместо объектов *struct sockaddr* используются объекты *struct sockaddr_in* (гнезда домена Internet), необходимо делать соответствующее преобразование типа.

При успешном выполнении эта функция возвращает 0, а в случае неудачи возвращает -1.

API Socket

int listen (int sid, int size);

Эта функция вызывается серверным процессом для создания гнезда, ориентированного на установление соединения.

Аргумент *sid* представляет собой дескриптор гнезда, возвращенный функцией *socket*.

Аргумент *size* задает максимальное число запросов на установление соединения, которые могут быть поставлены в очередь к данному гнезду.

При успешном выполнении эта функция возвращает 0, а в случае неудачи возвращает -1.

API Socket

int connect (int sid, struct sockaddr* addr_p, int len);

Эта функция вызывается в клиентском процессе для установления соединения с серверным гнездом.

Аргумент *sid* представляет собой дескриптор гнезда, возвращенный функцией *socket*. Вторым аргумент *addr_p* — это указатель на адрес объекта типа *struct sockaddr*. Для домена Internet структура адреса сокета имеет тип *struct sockaddr_in*. Должно быть выполнено соответствующее преобразование типа.

Аргумент *len* задает размер объекта (в байтах), на который указывает аргумент *addr_p*.

Если *sid* обозначает потоковое гнездо, то между клиентским и серверным гнездами устанавливается соединение с использованием виртуального канала. Потоковое гнездо клиента может соединяться с гнездом сервера только один раз. Если *sid* обозначает датаграммное гнездо, то для всех последующих вызовов функции *send*, осуществляемых через это гнездо, устанавливается адрес по умолчанию. Датаграммное гнездо может соединяться с гнездом сервера многократно, изменяя установленные по умолчанию адреса. Путем соединения с гнездом, имеющим NULL-адрес, датаграммные гнезда могут разорвать соединение.

При успешном выполнении эта функция возвращает 0, а в случае неудачи — 1.

API Socket

int accept (int sid, struct sockaddr* addr_p, int len_p);

Эта функция вызывается в серверном процессе для установления соединения с клиентским гнездом (которое делает запрос на установление соединения посредством вызова функции *connect*).

Аргумент *sid* представляет собой дескриптор гнезда, возвращенный функцией *socket*.

Аргумент *addr_p* — это указатель на адрес объекта типа *struct sockaddr*; в нем хранится имя клиентского гнезда, с которым устанавливает соединение серверное гнездо (тип *struct sockaddr_in*).

Аргумент *len_p* изначально устанавливается равным максимальному размеру объекта, указанному аргументом *addr_p*. При возврате он содержит размер имени клиентского гнезда, на которое указывает аргумент *addr_p*.

Если аргумент *addr_p* или аргумент *len_p* имеет значение NULL, эта функция не передает имя клиентского гнезда обратно в вызывающий процесс.

В случае неудачи рассматриваемая функция возвращает -1. В противном случае она возвращает дескриптор нового гнезда, с помощью которого серверный процесс может взаимодействовать исключительно с данным клиентом.

API Socket

int send (int sid, const char* buf, int len, int flag);

Эта функция передает содержащееся в аргументе *buf* сообщение длиной *len* байтов в гнездо, заданное аргументом *sid* и соединенное с данным гнездом.

Аргументу *flag* обычно присваивается значение 0, но он может иметь и значение *MSG_OOB*. В этом случае сообщение, содержащееся в *buf* должно быть передано как высокоприоритетное (out-of-band message).

В случае неудачи эта функция возвращает -1; в случае успешного выполнения возвращается число переданных байтов данных.

int sendto (int sid, const char* buf, int len, int flag, struct sockaddr* addr_p, int* len_p);

Эта функция делает то же самое, что и API *send*, только вызывающий процесс указывает также адрес гнезда-получателя (в аргументах *addr_p* и *len_p*).

Аргументы *sid*, *buf*, *len* и *flag* — те же самые, что в API *send*.

Аргумент *addr_p* — это указатель на объект, который содержит имя гнезда-получателя (тип *struct sockaddr_in*). Аргумент *len_p* содержит число байтов в объекте, на который указывает аргумент *addr_p*.

В случае неудачи данная функция возвращает -1; в случае успешного выполнения возвращается число переданных байтов данных.

API Socket

int recv (int sid, char* buf, int len, int flag);

Эта функция принимает сообщение через гнездо, указанное в аргументе *sid*. Принятое сообщение копируется в буфер *buf*, а максимальный размер *buf* задается аргументом *len*. Если в аргументе *flag* указан флаг *MSG_OOB*, то приему подлежит высокоприоритетное сообщение. В противном случае ожидается обычное сообщение. Кроме того, в аргументе *flag* может быть указан флаг *MSG_PEEK*, означающий, что процесс желает "взглянуть" на полученное сообщение, но не собирается удалять его из потокового гнезда. Такой процесс может повторно вызвать функцию *recv* для приема сообщения позже.

В случае неудачи функция *recv* возвращает -1; в случае успешного выполнения возвращается число байтов данных, записанных в буфер *buf*.

int recvfrom (int sid, const char* buf, int len, int flag, struct sockaddr* addr_p, int* len_p);

Эта функция делает то же самое, что и API *recv*, только при ее вызове задаются аргументы *addr_p* и *len_p*, позволяющие узнать имя гнезда-отправителя.

Аргументы *sid*, *buf*, *len* и *flag* — те же самые, что в API *recv*.

Аргумент *addr_p* — это указатель на объект, который содержит имя гнезда-отправителя (тип *struct sockaddr_in*).

Аргумент *len_p* сообщает число байтов в объекте, на который указывая аргумент *addr_p*.

API Socket

int shutdown (int sid, int mode);

Данная функция закрывает соединение между серверным и клиентским гнездами.

Аргумент *sid* — это дескриптор гнезда, возвращенный функцией *socket*. Аргумент *mode* задает режим закрытия.

Режим	Пояснение
0	Закрывает гнездо для чтения. При попытке продолжить чтение будут возвращаться нулевые байты (EOF)
1	Закрывает гнездо для записи. Дальнейшие попытки передать данные в это гнездо приведут к выдаче кода неудачного завершения, -1
2	Закрывает гнездо для чтения и записи. Дальнейшие попытки передать данные в это гнездо приведут к выдаче кода неудачного завершения -1, а при продолжении чтения будет возвращаться нулевое значение (EOF)

В случае неудачи данная функция возвращает -1, а в случае успешного выполнения — 0.

API Socket (вспомогательные функции)

gethostbyname

```
struct hostent
{
    char  *h_name;           /* Доменное имя хоста */
    char **h_aliases;        /* Псевдонимы */
    int    h_addrtype;       /* Тип адреса */
    int    h_length;         /* Длина адреса */
    char **h_addr_list;     /* Список адресов */
    #define h_addr h_addr_list[0]
}
```

Пример:

```
struct hostent *phe;
char *name = "csc.neic.nsk.su";

if (phe = gethostbyname(name))
{
    /* Теперь IP-адрес - в поле phe->h_addr */
}
else
    /* Ошибка в имени - обработать ошибку */ }
```

API Socket (getservbyname)

```
struct servent
{
    char    *s_name;           /* Стандартное имя службы */
    char    **s_aliases;      /* Псевдонимы */
    int      s_port;           /* Порт службы */
    char    *s_proto;         /* Используемый протокол */
}
```

Пример:

Пусть необходимо найти официально назначенный номер порта протокола для SMTP.

```
struct servent *pse;
```

```
if ( pse = getservbyname("smtp", "tcp"))
{
    /* Теперь номер порта - в поле pse->s_port */
}
else    /* Возникла ошибка - обработать ошибку */
```

API Socket (getprotobyname)

```
struct protoent
{
    char    *p_name;           /* Стандартное имя протокола */
    char **p_aliases;         /* Список допустимых псевдонимов */
    int     p_proto;          /* Стандартный номер протокола */
}
```

Пример:

Пусть необходимо найти официально назначенный номер порта протокола для SMTP.

```
struct protoent *ppe;

if ( ppe = getprotobyname("udp") )
{
    /* Теперь стандартный номер протокола в ppe->p_proto */
}
else    /* Возникла ошибка - обработать ошибку */
```

API Socket (Пример цикл для recv)

Пример:

```
#define BLEN 120          /* Длина используемого буфера */
char *req = "request of some sort";
char buf[BLEN] ;         /* Буфер для ответа */
char *bptr;              /* Указатель на буфер */
int n;                   /* Количество считанных байтов */
int buflen;              /* Место, оставшееся в буфере */
bptr = buf;
buflen = BLEN;

send(s, req, strlen(req), 0); /* Отправить запрос */
/* Получить ответ (может состоять из нескольких фрагментов) */

while(( n = recv(s, bptr, buflen, 0) ) > 0) {
    bptr+= n;
    buflen-= n;
}
```

Единственный вызов функции `send`, но предусматривает повторное выполнение вызовов функции `recv`. До тех пор, пока вызовы функции `recv` возвращают данные, в коде уменьшается счетчик пространства, доступного в буфере, а указатель буфера продвигается вслед за полученными данными. Такая итерация необходима даже если приложение на другом конце соединения передает каждый раз только небольшой объем данных, поскольку протокол TCP не гарантирует доставку их в виде таких же фрагментов, какие были отправлены.

API Socket

(Преобразование числа в сетевой/прямой порядок байт)

Преобразование числа
из **прямого** порядка
в **сетевой** порядок байт

int htons(int port)

Host Net Short

2 байта
0...65535

Преобразование числа
из **сетевого** порядка
в **прямой** порядок байт

int ntohs(int port)

Net Host Short

Преобразование числа
из **прямого** порядка
в **сетевой** порядок байт

int htonl(int addr4)

Host Net Long

4 байт
0... $2^{32} - 1$

Преобразование числа
из **сетевого** порядка
в **прямой** порядок байт

int ntohl(int addr4)

Net Host Long

Нахождение свободного порта

Функция `getsockname`

```
struct sockaddr_in  servAddr;
```

```
.....
```

```
servAddr.sin_family = AF_INET;
```

```
servAddr.sin_addr.s_addr = htonl( INADDR_ANY );
```

```
servAddr.sin_port = 0;
```

```
bind( sockMain, &servAddr, sizeof(servAddr));
```

```
getsockname( sockMain, &servAddr, &length );
```

```
printf( "СЕРВЕР: номер порта - % d\n", ntohs(servAddr.sin_port ) );
```

```
.....
```

Функции преобразования IP-адресов из строк ASCII в двоичные числа с сетевым порядком байтов (хранящиеся в структурах адресов сокетов) и обратно.

`inet_aton`

```
#include <arpa/inet.h>
```

```
int inet_aton(const char *strptr, struct in_addr *addrptr);
```

Возвращает: 1, если строка преобразована успешно, 0 в случае ошибки.

Функция преобразует строку, на которую указывает `strptr`, в 32-разрядное двоичное число (например, 172.103.15.189), записанное в сетевом порядке байтов, передаваемое через указатель `addrptr`. В случае успешного выполнения возвращаемое значение равно 1, иначе возвращается нуль.

Примечание. Функция `inet_aton` обладает одним недокументированным свойством: если `addrptr` — пустой указатель (`null pointer`), функция все равно выполняет проверку допустимости адреса, содержащегося во входной строке, но не сохраняет результата.

Функции преобразования IP-адресов.

`inet_addr`

```
in_addr_t inet_addr(const char *strptr) ;
```

Возвращает: 32-разрядный адрес IPv4 в сетевом порядке байтов: INADDR_NONE в случае ошибки.

Функция выполняет такое же преобразование, что и `inet_aton`, возвращая в качестве значения 32-разрядное двоичное число в сетевом порядке байтов. Проблема при использовании этой функции состоит в том, что все 2³² возможных двоичных значений являются действительными IP-адресами (от 0.0.0.0 до 255.255.255.255), но в случае возникновения ошибки функция возвращает константу `INADDR_NONE`.

Функция является нерекомендуемой, или устаревшей, и в создаваемом коде вместо нее следует использовать функцию `inet_aton`.

Функции преобразования IP-адресов.

`inet_ntoa`

Функция `inet_ntoa` является обратной к вышеописанным.

```
char *inet_ntoa(struct in_addr inaddr);
```

возвращает: указатель на строку с адресом в точечно-десятичной записи.

Функция преобразует 32-разрядный двоичный адрес IPv4, хранящийся в сетевом порядке байтов, в точечно-десятичную строку. Строка, на которую указывает возвращаемый функцией указатель, находится в статической памяти. Это означает, что функция не допускает повторного вхождения, то есть не является повторно входимой (reentrant). Наконец, отметим, что *эта функция принимает в качестве аргумента структуру, а не указатель на структуру.*

Функции преобразования IP-адресов. `inet_pton`

Более новые функции `inet_pton` и `inet_ntop` работают как с адресами IPv4, так и с адресами IPv6..

```
int inet_pton(int af, const char *src, void *dst);
```

Данная функция преобразует строку символов `src` в сетевой адрес (типа `af`), затем копирует полученную структуру с адресом в `dst`.

На текущий момент поддерживаются следующие типы адресов:

`AF_INET` – указывает на строку символов, содержащую сетевой адрес IPv4 в формате "ddd.ddd.ddd.ddd". Адрес преобразуется в `struct in_addr` и копируется в переменную `dst`, размер которой должен быть равен `sizeof(struct in_addr)` байтам.

`AF_INET6` – указывает на строку символов, содержащую адрес сети IPv6 в разрешенном для сети IPv6 формате адреса. Адрес преобразуется в `struct in6_addr` и копируется в переменную `dst`, размер которой должен быть `sizeof(struct in6_addr)` байтов.

ВОЗВРАЩАЕМЫЕ ЗНАЧЕНИЯ

`inet_pton` возвращает отрицательное значение и меняет значение переменной `errno` на `EAFNOSUPPORT`, если `af` не содержит правильного типа адреса. Возвращается 0, если `src` не содержит строку символов, представляющую правильный сетевой адрес (для указанного типа адресов). Если сетевой адрес был успешно преобразован, то возвращается положительное значение.

Функции преобразования IP-адресов. `inet_ntop`

```
const char *inet_ntop(int af, const void *src, char *dst, size_t cnt) ;
```

Данная функция преобразует структуру сетевого адреса `src` в строку символов с сетевым адресом (типа `af`), которая затем копируется в символьный буфер `dst`; размер этого буфера составляет `cnt` байтов.

На текущий момент **поддерживаются следующие типы адресов:**

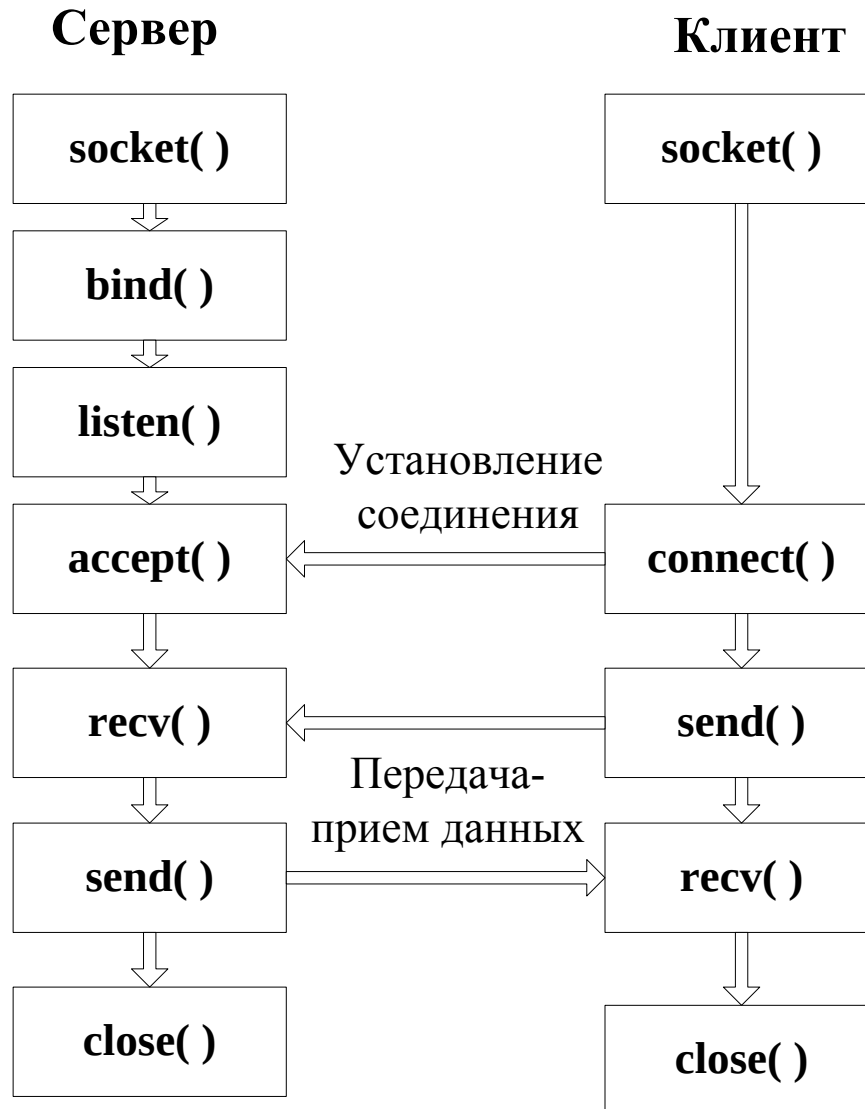
`AF_INET` (для `src`) указывает на `struct in_addr` (в формате сетевого порядка расположения байтов), которая преобразуется в IPv4-сетевой адрес в формате "ddd.ddd.ddd.ddd". Буфер `dst` должен быть размером, по меньшей мере, равным `INET_ADDRSTRLEN` байтам.

`AF_INET6` (для `src`) указывает на `struct in6_addr` (в формате сетевого порядка расположения байтов), которая преобразуется в представление этого адреса в наиболее верном формате IPv6. Буфер `dst` должен быть размером, по меньшей мере, равным `INET6_ADDRSTRLEN` байтам.

ВОЗВРАЩАЕМЫЕ ЗНАЧЕНИЯ

`inet_ntop` возвращает ненулевой указатель на `dst`. В случае ошибок возвращается `NULL`, а также `errno` присваивается значение `EAFNOSUPPORT`, если значение `af` не было установлено равным корректному типу адреса или равным `ENOSPC`, если полученная после преобразования строка превышает размер `dst` (заданный параметром `cnt`).

Программа типа клиент-сервер для ТСР



Программа сервера ТСР (начало)

```
#include <sys/types.h>      #include <sys/socket.h>      #include <stdio.h>
#include <netinet/in.h>      #include <netdb.h>          #include <errno.h>
```

```
main() {
    int sockMain, sockClient, length, child;
    struct sockaddr_in servAddr;

    if(( sockMain = socket( AF_INET, SOCK_STREAM, 0 ) ) < 0 ) {
        perror("Сервер не может открыть главный socket."); exit(1);
    }
    bzero( (char *) &servAddr, sizeof( servAddr ) );
    servAddr.sin_family = AF_INET;
    servAddr.sin_addr.s_addr = htonl( INADDR_ANY );
    servAddr.sin_port = 0;
    if ( bind( sockMain, &servAddr, sizeof(servAddr) ) ) {
        perror("Связывание сервера неудачно."); exit(1);
    }
    length = sizeof( servAddr );
    if ( getsockname( sockMain, &servAddr, &length ) ) {
        perror("Вызов getsockname неудачен."); exit(1);
    }
    printf( "СЕРВЕР: номер порта - % d\n", ntohs(servAddr.sin_port) )
    listen(sockMain, 5) ;
```


Программа сервера ТСР (продолжение)

```
for( ;; ) {
    if ( ( sockClient = accept( sockMain, 0, 0 ) ) < 0 ) {
        perror("Неверный socket для клиента."); exit(1);
    }
    BuffWork(sockClient);
    close(sockClient) ;
}

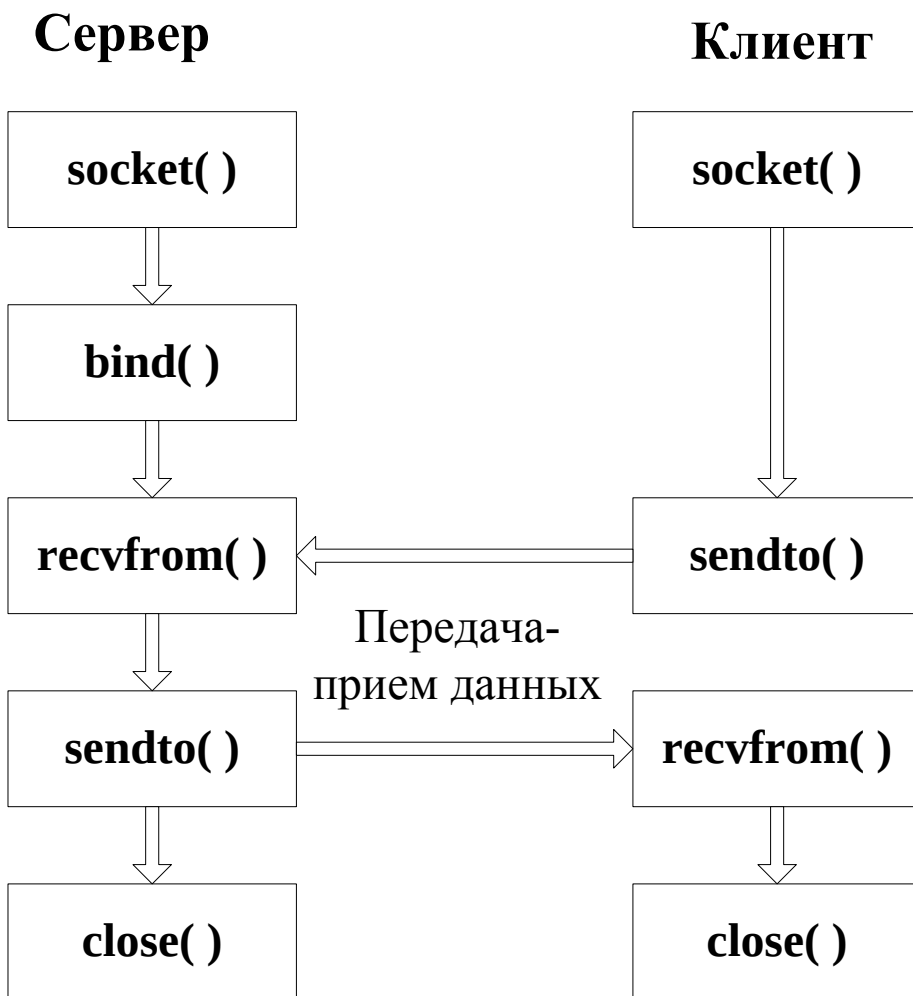
#define BUFLLEN 81

int BuffWork(int sockClient)
{
    char buf[BUFLLEN];
    int msgLength;
    bzero(buf, BUFLLEN);
    if( ( msgLength = recv( sockClient, buf, BUFLLEN, 0 ) ) < 0 ){
        perror("Плохое получение дочерним процессом."); exit(1);
    }
    printf( "SERVER: Socket для клиента - %d\n", sockClient);
    printf( "SERVER: Длина сообщения - %d\n", msgLength);
    printf( "SERVER: Сообщение: %s\n\n", buf);
}
```

Программа клиента ТСР

```
main( argc, argv )
int argc; char* argv [ ] ; {
    int sock;
    struct sockaddr_in servAddr;
    struct hostent *hp, *gethostbyname();
    if(argc < 4) { printf("ВВЕСТИ tcpclient имя_хоста порт сообщение\n"); exit(1) ;
        }
    if((sock = socket( AF_INET, SOCK_STREAM, 0 ) ) < 0) {
        perror("Не могу получить      socket\n"); exit(1);
    }
    bzero((char *)&servAddr, sizeof(servAddr) ) ;
    servAddr.sin_family = AF_INET;
    hp = gethostbyname( argv [1] ) ;
    bcopy( hp -> h_addr, &servAddr.sin_addr, hp -> h_length ) ;
    servAddr.sin_port = htons( atoi(argv[2]) ) ;
    if( connect( sock, &servAddr, sizeof( servAddr ) ) < 0 ){
        perror("Клиент не может соединиться.\n"); exit(1);
    }
    printf ("CLIENT: Готов к пересылке\n") ;
    if( send ( sock, argv[3], strlen(argv[3]), 0 ) < 0 ) {
        perror( "Проблемы с пересылкой.\n" ); exit(1);
    }
    printf("CLIENT: Пересылка завершена. Счастливо оставаться.\n");
    close(sock);  exit(0); }
```

Программа типа клиент-сервер для UDP



Программа сервера UDP (начало)

```
#include <sys/types.h>      #include <sys/socket.h>      #include <stdio.h>
#include <netinet/in.h>      #include <netdb.h>              #include <errno.h>
#define BUFLen 81
```

```
main() {
    int sockMain,  length,  msgLength ;
    struct sockaddr_in  servAddr,  clientAddr ;
    char  buf[BUFLen] ;

    if(( sockMain = socket( AF_INET, SOCK_DGRAM, 0 ) ) < 0 )
        {
            perror("Сервер не может открыть socket для UDP.");
            exit(1);
        }
    bzero( (char *) &servAddr,  sizeof( servAddr ) );

    servAddr.sin_family = AF_INET;
    servAddr.sin_addr.s_addr = htonl( INADDR_ANY );
    servAddr.sin_port = 0;

    if ( bind( sockMain, &servAddr, sizeof(servAddr) ) )
        {
            perror("Связывание сервера неудачно.");
            exit(1);
        }
}
```

Программа сервера UDP (продолжение)

```
length = sizeof( servAddr );
if ( getsockname( sockMain, &servAddr, &length ) )
    {
        perror("Вызов getsockname неудачен.");
        exit(1);
    }
printf( "СЕРВЕР: номер порта - % d\n", ntohs(servAddr.sin_port) )

for( ;; ) {
    length = sizeof( clientAddr );
    bzero( buf, sizeof( BUFLen) );

    if ( (msglength = recvfrom( sockMain, buf, BUFLen, 0, &clientAddr,
                                &length) ) < 0 )

        {
            perror("Плохой socket клиента.");
            exit(1);
        }

    printf( "SERVER: IP адрес клиента: %s\n", inet_ntoa(clientAddr.sin_addr) );
    printf( "SERVER: PORT клиента: %d\n", ntohs(clientAddr.sin_port) );
    printf( "SERVER: Длина сообщения - %d\n", msgLength);
    printf( "SERVER: Сообщение: %s\n\n", buf);
}
}
```

Программа клиента UDP

```
main( argc, argv )
    int argc;
    char* argv [ ] ;
{
    int sock;
    struct sockaddr_in  servAddr,  clientAddr ;
    struct hostent *hp, *gethostbyname();

    if(argc < 4)
        { printf( “ ВВЕСТИ udpclient имя_хоста порт сообщение\n”);
          exit(1) ;
        }

    if( (sock = socket( AF_INET, SOCK_DGRAM, 0 ) ) < 0)
        { perror("Не могу получить socket\n");
          exit(1);
        }

    bzero( (char *) &servAddr, sizeof(servAddr) ) ;

    servAddr.sin_family = AF_INET;
    hp = gethostbyname( argv [1] ) ;
    bcopy( hp -> h_addr, &servAddr.sin_addr, hp -> h_length ) ;
    servAddr.sin_port = htons( atoi(argv[2] ) ) ;
```

Программа клиента UDP (продолжение)

```
bzero( (char *)&clientAddr, sizeof(clientAddr) );
```

```
clientAddr.sin_family = AF_INET;
```

```
clientAddr.sin_addr.s_addr = htonl(INADDR_ANY) ;
```

```
clientAddr.sin_port = 0 ;
```

```
if ( bind( sock, &clientAddr, sizeof(clientAddr) ) )
```

```
{    perror(«Клиент не получил порт.»);
```

```
    exit(1);
```

```
}
```

```
printf("CLIENT: Готов к пересылке.\n");
```

```
if( sendto ( sock, argv[3], strlen(argv[3]), 0, &servAddr , sizeof(servAddr) ) < 0 )
```

```
{
```

```
    perror( "Проблемы с sendto . \n" );
```

```
    exit(1);
```

```
}
```

```
printf("CLIENT: Пересылка завершена. Счастливо оставаться. \n");
```

```
close(sock);
```

```
}
```

Функция fork()

https://www.opennet.ru/docs/RUS/linux_parallel/node7.html

<https://ru.wikipedia.org/wiki/Fork>

Системный вызов `fork()` создаёт новый процесс. Синтаксис вызова следующий:

```
#include <sys/types>
#include <unistd.h>
```

```
pid_t fork(void);
```

`pid_t` является примитивным типом данных, который **определяет идентификатор процесса** или группы процессов. При вызове `fork()` порождается новый процесс (процесс-потомок), который почти идентичен порождающему процессу-родителю.

Процесс-потомок наследует следующие признаки родителя:

- сегменты кода, данных и стека программы;
- таблицу файлов, в которой находятся состояния флагов дескрипторов файла, указывающие, читается ли файл или пишется. Кроме того, в таблице файлов содержится текущая позиция указателя записи-чтения;
- рабочий и корневой каталоги;
- реальный и эффективный номер пользователя и номер группы;
- приоритеты процесса (администратор может изменить их через `nice`);
- контрольный терминал;
- маску сигналов;
- ограничения по ресурсам;
- сведения о среде выполнения;
- разделяемые сегменты памяти.

Потомок не наследует от родителя следующих признаков:

- идентификатора процесса (PID, PPID);
- израсходованного времени ЦП (оно обнуляется);
- сигналов процесса-родителя, требующих ответа;
- заблокированных файлов (record locking).

Функция `fork()`

При успешном вызове `fork()` в системе выполняются, два почти «идентичных» процесса. Весь код после `fork()` выполняется, как в процессе-потомке, так и в процессе-родителе.

```
PID = fork();
```

```
If PID < 0; /* Ошибка при порождении процесса. */
```

Процесс-потомок и процесс-родитель получают разные коды возврата после вызова `fork()`.

```
If PID > 0; /* Это процесс-родитель, который успешно создал процесс-потомок.  
Значение записанное в PID – идентификатор созданного процесса-потомка. */
```

```
If PID == 0; /* Это процесс-потомок.
```

Процесс-потомок получает в качестве кода возврата значение 0, если вызов `fork()` оказался успешным. ***/**

Применение процессов для обеспечения параллельной работы сервера

- `/* Создаем ведущий сокет, привязываем его к общепринятому порту и переводим в пассивный режим */`
- `for( ; ; ) {`
- `if( sockClient = accept( sockMain, (struct sockaddr *)&fsin, &len) ) < 0 )`
- `{ /* Сообщение об ошибке */`
- `exit(1);`
- `}`
- `child = fork() ;`
- `if( child < 0 )`
- `{ /* Сообщение об ошибке */`
- `exit(1);`
- `}`
- `if( child == 0 )`
- `{`
- `close( sockMain ) ;`
- `/* Обработка поступившего запроса процессом */`
- `close( sockClient ) ;`
- `exit( 0 ) ;`
- `}`
- `close( sockClient ) ;`
- `}`

Применение процессов для обеспечения параллельной работы сервера

Родительский процесс

```
...  
for (;;) {  
    sn=accept(s,...);  
    f=fork();  
    If(f==0) {  
        ...  
    }  
    If(f>0) {  
        Close(sn);  
        ... }  
    ... }  
}
```

Child for client1

```
...  
for (;;) {  
    sn=accept(s,...);  
    f=fork();  
    If(f==0) {  
        Close(s);  
        Send(sn,...);  
        Recv(sn,...);  
        ...  
        Close(sn);  
        Exit(0);  
    }  
    If(f>0) {  
        ... }  
}
```

Child for client2

```
...  
for (;;) {  
    sn=accept(s,...);  
    f=fork();  
    If(f==0) {  
        Close(s);  
        Send(sn,...);  
        Recv(sn,...);  
        ...  
        Close(sn);  
        Exit(0);  
    }  
    If(f>0) {  
        ... }  
}
```

Child for client3

```
...  
for (;;) {  
    sn=accept(s,...);  
    f=fork();  
    If(f==0) {  
        Close(s);  
        Send(sn,...);  
        Recv(sn,...);  
        ...  
        Close(sn);  
        Exit(0);  
    }  
    If(f>0) {  
        ... }  
}
```

Применение процессов для обеспечения параллельной работы сервера (Зомби процессы)

О наличии зомби-процесса можно узнать командой: `>ps -aux`

```
kirill 9097 0.0 0.0 10856 3332 pts/6 R+ 11:20 0:00 ps -aux
kirill@kirill:~$ ps -aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   1816   1184 ?        SL   09:35    0:00 /init
root         7  0.0  0.0   1812     88 ?        Ss   09:35    0:00 /init
root         8  0.0  0.0   1812     96 ?        S    09:35    0:00 /init
kirill        9  0.0  0.1  10272   5380 pts/0    Ss   09:35    0:00 -bash
root        291  0.0  0.0   1816    100 ?        Ss   09:36    0:00 /init
root        292  0.0  0.0   1824    100 ?        S    09:36    0:00 /init
kirill       293  0.0  0.1  10272   5328 pts/4    Ss+  09:36    0:00 -bash
root        321  0.0  0.0   1816    100 ?        Ss   09:36    0:00 /init
root        322  0.0  0.0   1824    100 ?        S    09:36    0:00 /init
kirill       323  0.0  0.1  10272   5352 pts/5    Ss+  09:36    0:00 -bash
kirill       772  0.0  0.0   5884   1568 pts/0    T    09:41    0:00 ./server
root       1057  0.0  0.0   1816    100 ?        Ss   09:44    0:00 /init
root       1058  0.0  0.0   1824    100 ?        S    09:44    0:00 /init
kirill      1059  0.0  0.1  10168   4992 pts/6    Ss   09:44    0:00 -bash
root       1609  0.0  0.0   1816    100 ?        Ss   09:50    0:00 /init
root       1610  0.0  0.0   1824    100 ?        S    09:50    0:00 /init
kirill      1611  0.0  0.0   2612    524 pts/1    Ss+  09:50    0:00 sh -c "$VSCODE_WSL_EXT_LOCATION/scripts/wslServer
kirill      1612  0.0  0.0   2612    592 pts/1    S+   09:50    0:00 sh /mnt/c/Users/grand/.vscode/extensions/ms-vscode
kirill      1617  0.0  0.0   2612    592 pts/1    S+   09:50    0:00 sh /home/kirill/.vscode-server/bin/5c3e652f63e798
kirill      1621  0.2  2.7 9600004 94824 pts/1    Sl+  09:50    0:13 /home/kirill/.vscode-server/bin/5c3e652f63e798a5a
root       1632  0.0  0.0   1828    100 ?        Ss   09:50    0:00 /init
root       1633  0.0  0.0   1828    112 ?        S    09:50    0:00 /init
kirill      1634  0.0  1.7 6088852 60892 pts/2    Ssl+ 09:50    0:01 /home/kirill/.vscode-server/bin/5c3e652f63e798a5a
root       1641  0.0  0.0   1828    100 ?        Ss   09:50    0:00 /init
root       1642  0.0  0.0   1828    112 ?        S    09:50    0:01 /init
kirill      1643  0.0  1.4 599636 51984 pts/3    Ssl+ 09:50    0:03 /home/kirill/.vscode-server/bin/5c3e652f63e798a5a
kirill      1655  0.0  1.5 851224 55200 pts/1    Sl+  09:50    0:01 /home/kirill/.vscode-server/bin/5c3e652f63e798a5a
kirill      1682  6.3  4.7 1034952 163600 pts/1    Sl+  09:50    5:46 /home/kirill/.vscode-server/bin/5c3e652f63e798a5a
kirill      1719  0.1  1.7 112676 61664 pts/1    Sl+  09:50    0:08 /home/kirill/.vscode-server/extensions/ms-vscode.
kirill     1804  0.0  0.0   2492    508 pts/0    T    09:50    0:00 ./server
kirill     3597  0.0  0.7 4261588 26208 pts/1    Sl+  10:12    0:01 /home/kirill/.vscode-server/extensions/ms-vscode.
kirill     8472  0.0  0.4 4260940 16568 pts/1    Sl+  11:13    0:00 /home/kirill/.vscode-server/extensions/ms-vscode.
kirill     8684  0.0  0.0   2492    504 pts/0    T    11:16    0:00 ./server
kirill     9034  0.0  0.0   2492    508 pts/0    S+   11:20    0:00 ./server
kirill     9071  0.0  0.0      0      0 pts/0    Z+   11:20    0:00 [server] <defunct>
kirill     9079  0.0  0.0      0      0 pts/0    Z+   11:20    0:00 [server] <defunct>
kirill     9119  0.0  0.0  10856   3348 pts/6    R+   11:21    0:00 ps -aux
kirill@kirill:~$
```

Применение процессов для обеспечения параллельной работы сервера (зомби-процесс)

Пример 2

- **signal(SIGCHLD, reaper) ;**
- **while(1) {**
- **ssock = accept(msock, (struct sockaddr *)&fsin, &len) ;**
- **if(ssock < 0){ /* Сообщение об ошибке */ }**
- **switch(fork()) {**
- **case(0) :**
- **close(msock) ;**
- **/* Обработка поступившего запроса ведомым процессом */**
- **close(ssock) ;**
- **exit(0) ;**
- **default:**
- **close(ssock) ;**
- **/* Ведущий процесс */**
- **break;**
- **case (-1) :**
- **/* Сообщение об ошибке */**
- **}**
- **}**

void reaper(int sig)

- **{ int status;**
- **while(wait3(&status, WNOHANG, (struct rusage *) 0) >= 0) ;**
- **}**

Применение процессов для обеспечения параллельной работы сервера

```
signal( SIGCHLD, reaper );
```

```
for(;;) { ....  
}
```

```
void reaper( int sig ){ int status;  
while( wait3( &status, WNOHANG,  
(struct rusage *) 0 ) >= 0 );
```

Родительский процесс

```
...  
for(;;) {  
sn=accept( s,... );  
f=fork();  
If (f==0) {  
...  
}  
If (f>0) {  
Close( sn );  
... }  
... }
```

*Можно узнать о
наличии зомби-процесса
командой: >ps -aux*

Child for client1

```
...  
for(;;) {  
sn=accept( s,... );  
f=fork();  
If (f==0) {  
Close( s );  
Send( sn,... );  
Recv( sn,... );  
...  
Close( sn );  
Exit( 0 );  
}  
If (f>0) {  
... }
```

Child for client2

```
...  
for(;;) {  
sn=accept( s,... );  
f=fork();  
If (f==0) {  
Close( s );  
Send( sn,... );  
Recv( sn,... );  
...  
Close( sn );  
Exit( 0 );  
}  
If (f>0) {  
... }
```

Child for client3

```
...  
for(;;) {  
sn=accept( s,... );  
f=fork();  
If (f==0) {  
Close( s );  
Send( sn,... );  
Recv( sn,... );  
...  
Close( sn );  
Exit( 0 );  
}  
If (f>0) {  
... }
```

Применение процессов для обеспечения параллельной работы сервера (зомби-процесс)

Замечание: первоначальный, и новые процессы имеют доступ к открытым сокетам после вызова функции **fork()** и они оба должны закрыть эти сокеты, после чего система освобождает связанные с ним ресурсы.

Завершившийся процесс остается в виде так называемого процесса-зомби до тех пор, пока родительским процессом не будет выполнен системный вызов **wait3**. Для полного завершения дочернего процесса (т.е. уничтожения процесса-зомби) необходимо перехватывается сигнал завершения дочернего процесса. Операционной системе дается указание, что для ведущего процесса сервера при получении каждого сигнала о завершении работы дочернего процесса (сигнал **SIGCHLD**) должна быть выполнена функция **reaper**, в виде следующего вызова: **signal(SIGCHLD, reaper);**

Функция **reaper** вызывает системную функцию **wait3**, которая остается заблокированной до тех пор, пока не произойдет завершение работы одного или нескольких дочерних процессов. Эта функция возвращает значение структуры **status**, которую можно проанализировать для получения дополнительной информации о завершившемся процессе. Поскольку данная программа вызывает функцию **wait3** при получении сигнала **SIGCHLD**, вызов этой функции всегда происходит только после завершения работы дочернего процесса. Для предотвращения возникновения в сервере тупиковой ситуации в случае ошибочного вызова в программе используется параметр **WNOHANG**, который указывает, что функция **wait3** не должна блокироваться в ожидании завершения какого-либо процесса. *Можно узнать о наличии зомби-процесса командой:* `>ps -aux`

Если в обработчике **SIGCHLD** нет ничего кроме вызова **wait**, то можно просто установить этот обработчик в **SIG_IGN**. Тут следует заметить, что установка **SIGCHLD** в **SIG_IGN** совместима с POSIX.1-2001, и не совместима с POSIX.1-1990.

Применение потоков для обеспечения параллельной работы сервера

- **int pthread_create(pthread_t *thread, pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);**

После создания нового потока в нем начинается выполняться функция (которая называется потоковой функцией), переданная параметром **start_routine**, причем ей самой в качестве первого параметра передается переменная **arg**. Параметр **attr** позволяет задать атрибуты потока (**NULL** для значений по умолчанию). **thread** -- адрес переменной, в которую **pthread_create()** записывает идентификатор созданного потока. Созданный с помощью **pthread_create()** поток будет работать параллельно с существующими. Возвращается 0 в случае успеха и не ноль -- в противоположном случае. Потоковая функция **start_routine** имеет прототип:

- **void* my_thread_function(void *);**

Поскольку как параметр, так и ее возвращаемое значение -- указатели, то функция может принимать в качестве параметра и возвращать любую информацию.

Применение потоков для обеспечения параллельной работы сервера

- Выполнение потока завершается в двух случаях: если завершено выполнение потоковой функции, или при выполнении функции **pthread_exit()**:
- **void pthread_exit(void *retval);**

которая завершает выполнение вызвавшего ее потока. Аргумент **retval** -- это код с которым завершается выполнение потока. При завершении работы потока вы должны помнить, что **pthread_exit()** не закрывает файлы и все открытые потоком файлы будут оставаться открытыми даже после его завершения, так что не забывайте подчищать за собой. Если вы завершите выполнение функции **main()** с помощью **pthread_exit()**, выполнение порожденных ранее потоков продолжится.

Применение потоков для обеспечения параллельной работы сервера

Родительский поток

```
...  
for (;;) {  
    sn=accept (s,...) ;  
    pthread_create  
    (... ,func_client,sn ) ;  
    ...  
}  
...
```

Thread for client1

```
func(void sn) {  
    ...  
    Send (sn...) ;  
    Recv (sn...) ;  
    ...  
}
```

Thread for client2

```
func(void sn) {  
    ...  
    Send (sn...) ;  
    Recv (sn...) ;  
    ...  
}
```

Thread for client3

```
func(void sn) {  
    ...  
    Send (sn...) ;  
    Recv (sn...) ;  
    ...  
}
```

Применение потоков для обеспечения параллельной работы сервера

- Потоки, как и порожденные процессы, по завершению работы сами по себе не освобождают ресурсы, занятые собой для личного пользования (а именно дескриптор и стек) :-). Поэтому им необходимо помочь.

Варианта, собственно, два: либо на ряду с освобождением ресурсов какой-либо поток ждет его завершения, либо нет.

Для первого варианта используем функцию **pthread_join()**:

- **int pthread_join(pthread_t th, void **thread_return);**

которая приостанавливает выполнение вызвавшего ее процесса до тех пор, пока поток, определенный параметром **th**, не завершит свое выполнение и если параметр **thread_return** не будет равен **NULL**, то запишет туда возвращенное потоком значение (которое будет равным либо **PTHREAD_CANCELED**, если поток был отменен, либо тем значением, которое было передано через аргумент функции **pthread_exit()**).

Применение потоков для обеспечения параллельной работы сервера

- Для второго варианта есть функция **pthread_detach()** :
- **int pthread_detach(pthread_t th);**

которая делает поток **th** "открепленным" (detached). Это значит, что после того, как он завершится, он сам освободит все занятые ним ресурсы.

Обратите внимание на то, что нельзя ожидать завершения detached-потока (то есть функция **pthread_join** выполненная для detached потока завершится с ошибкой).

Применение потоков для обеспечения параллельной работы сервера

- Создание потоков позволяет им совместно использовать некоторые ресурсы, но нужно производить контроль на предмет эксклюзивности доступа к этим ресурсам (нельзя допускать одновременной записи в одну переменную, например).

Такой контроль можно вести тремя способами через:

- *взаимоисключающую блокировку*
- *условные переменные*
- *Семафоры*

Применение потоков для обеспечения параллельной работы сервера

- Первый вариант представляется самым набором функций библиотеки **POSIX Threads**. Взаимоисключающая блокировка представляется в программе переменной типа **pthread_mutex_t**.
Для работы с ними существует 5 функций, а именно:
 - **int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);**
которая инициализирует блокировку, заданную параметром **mutex**. Соответственно, **mutexattr** -- ее атрибуты. Значение **NULL** соответствует установкам по умолчанию.
 - **int pthread_mutex_destroy(pthread_mutex_t *mutex);**
удаляет блокировку **mutex**

Применение потоков для обеспечения параллельной работы сервера

- **int pthread_mutex_lock(pthread_mutex_t *mutex)**

устанавливает блокировку **mutex**. Если **mutex** не была заблокирована, то она его и немедленно завершается. Если же нет, то функция приостанавливает работы вызвавшего ее потока до разблокировки **mutex**, а после этого выполняет аналогичные действия.

- **int pthread_mutex_unlock(pthread_mutex_t *mutex)**

снимает блокировку **mutex**. Подразумевается, что эта функция будет вызвана тем же потоком, который ее заблокировал (через **pthread_mutex_lock()**).

- **int pthread_mutex_trylock(pthread_mutex_t *mutex)**

ведет себя аналогично **pthread_mutex_lock()** за исключением того, что она не приостанавливает вызывающий поток, если блокировка **mutex** установлена, а просто завершается с кодом **EBUSY**.

Применение потоков для обеспечения параллельной работы сервера

- Поток выполнения представляет собой один из принципов организации отдельных вычислений, а один процесс может содержать от одного и более потоков.
- Новый поток может быть создан в любое время путем вызова функции **pthread_create**.
- Операционная система ограничивает максимально допустимое количество параллельных потоков, также как и максимальное количество параллельных процессов.
- Все потоки процесса разделяют единый набор глобальных переменных и единый набор дескрипторов файлов.
- Многопоточковые процессы обладают двумя основными **преимуществами** по сравнению с однопоточковыми процессами:
- ***более высокая эффективность и разделяемая память.***

Повышение эффективности связано с уменьшением издержек на переключение контекста.

Применение потоков для обеспечения параллельной работы сервера

- *Переключение контекста* —

это действия, выполняемые операционной системой при передаче ресурсов процессора от одного потока выполнения к другому.

При переключении с одного потока на другой операционная система должна сохранить в памяти состояние предыдущего потока (например, значения регистров) и восстановить состояние следующего потока.

Потоки в одном и том же процессе разделяют значительную часть информации о состоянии процесса, поэтому операционной системе приходится выполнять меньший объем работы по сохранению и восстановлению состояния.

Вследствие этого переключение с одного потока на другой в одном и том же процессе происходит быстрее по сравнению с переключением между двумя потоками в разных процессах.

Применение потоков для обеспечения параллельной работы сервера

- Второе преимущество потоков (*разделяемая память*), является более важным, чем повышение эффективности.

Потоки упрощают разработку параллельных серверов, в которых все копии сервера должны взаимодействовать друг с другом или обращаться к разделяемым элементам данных. В частности, поскольку ведомые потоки в сервере совместно используют глобальную память.

Одним *из недостатков потоков является* то, что они имеют *общее состояние процесса*, поэтому действия, выполненные одним потоком, могут повлиять на другие потоки в том же процессе. Например, если два потока попытаются одновременно обратиться к одной и той же переменной, они могут помешать друг другу. API-интерфейс потоков предоставляет функции, которые могут использоваться потоками для координации работы.

Применение потоков для обеспечения параллельной работы сервера

- Еще один *недостаток* связан с *отсутствием надежности*.

Если одна из параллельно работающих копий однопоточкового сервера вызовет серьезную ошибку (например, в ней будет выполнена ссылка на недопустимую область памяти), то операционная система завершит только тот процесс, который вызвал ошибку. С другой стороны, если серьезная ошибка будет вызвана одним из потоков многопоточкового сервера, то операционная система завершит весь процесс (т.е. все потоки этого процесса).

Пример сервера, реализованного с применением потоков

- `pthread_mutex_t st_mutex; /* Разделяемая переменная */`
- `int main() {`
- `pthread_t th;`
- `pthread_attr_t ta;`
- `/* Создаем ведущий сокет, привязываем его к общепринятому порту и`
- `переводим в пассивный режим */`
- `pthread_attr_init(&ta);`
- `pthread_attr_setdetachstate(&ta, PTHREAD_CREATE_DETACHED);`
- `pthread_mutex_init(&st_mutex, 0);`
- `while (1) {sock = accept(msock, (struct sockaddr *)&fsin, &len);`
- `if (sock < 0) {/* ошибка */}`
- `if (pthread_create( &th, &ta, (void *)handler, (void *) sock) < 0 )`
- `{ /* ошибка */}`
- `}`

`int handler ( int sock )`

- `{ pthread_mutex_lock(&st_mutex);`
- `/* выполнение операций с разделяемыми переменными */`
- `pthread_mutex_unlock(&st_mutex);`
- `return 0;`
- `}`

Применение потоков для обеспечения параллельной работы сервера

- Аргумент **attr** содержит атрибуты, присваиваемые вновь создаваемому потоку. Значение аргумента может быть равно **NULL**, если новый поток должен использовать атрибуты, принятые системой по умолчанию, или адрес объекта содержит атрибуты. Объект, содержащий атрибуты, может быть связан с несколькими потоками.
- Функция **pthread_attr_init** создает объект, содержащий атрибуты, а функция **pthread_attr_destroy** удаляет такой объект:

```
#include <pthread.h>
```

```
Int pthread_attr_init(pthread_attr_t* attr_p);
```

```
Int pthread_attr_destroy(pthread_attr_t* attr_p);
```

- Атрибуты объекта, созданного функцией **pthread_attr_init**, можно проверить функцией **pthread_attr_get**, или установить функцией **pthread_attr_set**.
- Например, состояние отсоединения

API для проверки - **pthread_attr_getdetachstate**

API для установки - **pthread_attr_setdetachstate**

Применение потоков для обеспечения параллельной работы сервера

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define NTHRDS 2 /* К этим ресурсам мы можем получить доступ из потока */

char words[8000][20]; // Словарь
int current, maxw;
char foundpass; // Флажок

pthread_mutex_t mutexpass, mutexfound; /* Блокировки */

void *passhack(void *hash)

int main(int argc, char **argv)
{
    char i, *pass;
    FILE *passfile;
```

Применение потоков для обеспечения параллельной работы сервера

```
pthread_t thrds[NTHRDS]; // Потоки
pass = (char *)malloc(35);
pthread_mutex_init(&mutexpass, NULL); /* Инициализация блокировок */
pthread_mutex_init(&mutexfound, NULL);
    strcpy(pass, argv[1]); /* Читаем словарь в память */
    strcpy(pass, argv[1]); /* Читаем словарь в память */
    passfile = fopen(argv[2], "r");
    maxw = 0;
    while ( !feof(passfile)) fgets(words[maxw++], 20, passfile);
        fclose(passfile);
    foundpass = 0;
/* Запускаем потоки */
for ( i=0 ; i < NTHRDS; i++ )
    pthread_create(&thrds[i], NULL, passhack, (void *) pass);
        /* И ждем завершения их работы */
    for ( i=0 ; i < NTHRDS; i++ )
        pthread_join(thrds[i], NULL);

/* Освобождаем блокировки */
pthread_mutex_destroy(&mutexpass);
pthread_mutex_destroy(&mutexfound);
    return 0;
}
```

Применение потоков для обеспечения параллельной работы сервера

- Аргумент **attr** содержит атрибуты, присваиваемые вновь создаваемому потоку. Значение аргумента может быть равно **NULL**, если новый поток должен использовать атрибуты, принятые системой по умолчанию, или адрес объекта содержит атрибуты. Объект, содержащий атрибуты, может быть связан с несколькими потоками.
- Функция **pthread_attr_init** создает объект, содержащий атрибуты, а функция **pthread_attr_destroy** удаляет такой объект:

```
#include <pthread.h>
```

```
Int pthread_attr_init(pthread_attr_t* attr_p);
```

```
Int pthread_attr_destroy(pthread_attr_t* attr_p);
```


Применение потоков для обеспечения параллельной работы сервера

- Атрибуты объекта, созданного функцией **pthread_attr_init**, можно проверить функцией **pthread_attr_get**, или установить функцией **pthread_attr_set**.
- Например,

состояние отсоединения

API для проверки - **pthread_attr_getdetachstate**

API для установки - **pthread_attr_setdetachstate**

правила планирования

API для проверки - **pthread_attr_getschedpolicy**

API для установки – **pthread_attr_setschedpolicy**

Правила планирования задают, среди прочего, приоритет потока

Параметры планирования

API для проверки - **pthread_attr_getschedparam**

API для установки - **pthread_attr_setschedparam**

*Второй аргумент в **pthread_attr_getschedparam** и **pthread_attr_setschedparam** – это адрес переменной типа **struct sched_param**. В этой переменной есть целочисленное поле **sched_priority**, в котором задается приоритет любого потока, обладающего этим свойством.*

Однопотокковые псевдопараллельные серверы

- Необходимо предусмотреть, чтобы единственный поток выполнения в сервере держал открытыми соединения с несколькими клиентами и обеспечивал обслуживание сервером того соединения, через которое в определенный момент поступают данные.
- Однопотковая реализация не требует переключения между контекстами потоков или процессов, поэтому она может выдерживать более высокую нагрузку по сравнению с реализацией, в которой используются несколько потоков или процессов.

Асинхронный ввод/вывод, организованный с помощью системного вызова **select**

- В основе разработки программы однопоточкового, параллельного сервера лежит использование асинхронного ввода/вывода, организованного с помощью системного вызова **select**.
- Сервер создает сокет для каждого соединения, которое он должен поддерживать, а затем вызывает функцию **select**, которая ожидает поступления данных через каждое из них.
- Функция **select** может ожидать поступления запросов на выполнение операций ввода/вывода через все возможные сокеты, в том числе и одновременно ожидать поступления новых запросов на установление соединения.

Асинхронный ввод/вывод, организованный с помощью системного вызова `select`

- **НАЗВАНИЕ**

`select`, `FD_CLR`, `FD_ISSET`, `FD_SET`, `FD_ZERO` - синхронное мультиплексирование ввода-вывода

- **КРАТКАЯ СВОДКА**

```
#include <sys/time.h>
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

- **`int select(int n, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);`**

```
FD_CLR(int fd, fd_set *set);
```

```
FD_ISSET(int fd, fd_set *set);
```

```
FD_SET(int fd, fd_set *set);
```

```
FD_ZERO(fd_set *set);
```

Асинхронный ввод/вывод, организованный с помощью системного вызова `select`

ОПИСАНИЕ

- **`select`** ждет изменения статуса нескольких файловых дескрипторов. Отслеживаются три независимых набора дескрипторов. Те, что перечислены в параметре *readfds*, будут отслеживаться на предмет появления новых символов, доступных для чтения (говоря точнее, операция чтения не будет блокирована -- в частности, файловый дескриптор находится в конце файла); те, что указаны в параметре *writefds*, будут отслеживаться на предмет того, что операция записи не будет заблокирована; те же, что указаны в параметре *exceptfds*, будут отслеживаться на предмет исключительных ситуаций. При возврате из функции наборы дескрипторов модифицируются, чтобы показать, какие из них изменили свой статус.
- Для манипуляций наборами существуют четыре макроса: **`FD_ZERO`** очищает набор. **`FD_SET`** и **`FD_CLR`** добавляют или удаляют заданный дескриптор из набора. **`FD_ISSET`** проверяет, является ли дескриптор частью набора; этот макрос полезен после возврата из функции **`select`**.
- *n* на единицу больше самого большого номера дескриптора из всех наборов.
- **`timeout`** -- это верхняя граница времени, которое пройдет перед возвратом из **`select`**. Можно использовать ноль, при этом **`select`** завершится немедленно. Если **`timeout`** равен **`NULL`** (нет таймаута), то **`select`** будет ожидать изменений неопределенное время.

Пример. Системный вызов select

ПРИМЕР

```
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

int main ( void )
{ fd_set rfd;
  struct timeval tv;
  int retval;
  /* Ждем, пока на стандартном вводе (fd 0) что-нибудь появится. */
  FD_ZERO(&rfd);
  FD_SET(0, &rfd);
  /* Ждем не больше пяти секунд. */
  tv.tv_sec = 5;
  tv.tv_usec = 0;
  retval = select(1, &rfd, NULL, NULL, &tv); /* Не полагаемся на значение tv! */
  if (retval)
    printf("Данные доступны.\n"); /* Теперь FD_ISSET(0, &rfd) вернет истинное значение. */
  else
    printf("Данные не появились в течение пяти секунд.\n");
  exit(0);
}
```

Однопоточковые псевдопараллельные серверы

```
int msock; /* Ведущий сокет сервера */
fd_set rfd; /* Набор дескрипторов, готовых к чтению */
fd_set afd; /* Набор активных дескрипторов */
int fd, nfds, ssock ;

    /* инициализация пассивного сокета msock */

    nfds = getdtablesize();
    FD_ZERO(&afd);  FD_SET(msock, &afd);
while (1) {
    memcpy(&rfd, &afd, sizeof(rfd));
    if (select(nfds, &rfd, (fd_set *)0, (fd_set *)0, (struct timeval *)0) < 0) { /* ошибка */}

        /*          Блок 1          подключение клиента          */

    if ( FD_ISSET(msock, &rfd) ) {      alen = sizeof(fsin);
        ssock = accept(msock, (struct sockaddr *)&fsin, &alen); if (ssock < 0) { /* ошибка */}
        FD_SET(ssock, &afd);
    }

        /*          Блок 2          обработка запросов клиентов          */

    for ( fd = 0 ; fd < nfds ; fd++ )
        if ( fd != msock && FD_ISSET(fd, &rfd) )
            if ( handler(fd) == 0 ) { /* число полученных байт */
                close(fd);
                FD_CLR(fd, &afd);
            }
    }
```

Однопоточковые псевдопараллельные серверы

ВОЗВРАЩАЕМОЕ ЗНАЧЕНИЕ

- При успешном завершении **select** возвращает количество дескрипторов, находящихся в наборах дескрипторов, причем это количество может быть равным нулю, если таймаут истекает, а интересующие нас события так и не произошли. При ошибке возвращается -1, а *errno* устанавливается должным образом; наборы дескрипторов и значение *timeout* становятся неопределены, поэтому при ошибке нельзя полагаться на их значение.

ОШИБКИ

EBADF В одном из наборов находится неверный файловый дескриптор.

EINTR Был пойман незаблокированный сигнал.

EINVAL *n* отрицательно.

ENOMEM Функция **select** не смогла выделить участок памяти для внутренних таблиц.

ЗАМЕЧАНИЕ

- В некоторых программах **select** вызывается с тремя пустыми наборами файлов, *n* равным нулю, и ненулевым значением *timeout*, что является довольно переносимым способом сделать задержку с миллисекундной точностью.
- Под Linux *timeout* изменяется, чтобы сообщить количество времени, которое не было использовано; большинство других реализаций не делают этого. Это приводит к проблемам как в коде под Linux, который читает значение *timeout* и переносится в другие операционные системы, так и когда код переносится под Linux и использует при этом *struct timeval* для нескольких функций **select** в цикле без повторной инициализации. Считайте, что параметр *timeout* неопределен после возврата из функции **select**.

Мультипротокольный сервер (TCP, UDP)

```
int      tsock; /* Ведущий сокет TCP */
int      usock; /* Сокет UDP */
int      ssock;  /* Ведомый сокет TCP */
fd_set   rfdset; /* Дескрипторы, готовые к чтению */
/* инициализация сокетов tsock и usock */
nfdset = MAX(tsock, usock) + 1; /*Длина битовой маски для набора дескрипторов */
FD_ZERO(&rfdset);
while (1) {
    FD_SET(tsock, &rfdset); FD_SET(usock, &rfdset);
    if (select(nfdset, &rfdset, (fd_set *)0, (fd_set *)0, (struct timeval *)0)
<0) { /* ошибка */}
    if (FD_ISSET(tsock, &rfdset)) {
        len=sizeof(fsin);
        ssock = accept(tsock, (struct sockaddr*)&fsin,&len); if(ssock<0){
/*ошибка*/}
        handler(ssock, buf); /* обработчик tcp клиента основан на
параллельном потоке или процессе*/
        .....
    }
    if (FD_ISSET(usock, &rfdset)) {
        len = sizeof(fsin);
        if (recvfrom(usock, buf, sizeof(buf), 0, (struct sockaddr *)&fsin,
&len) <0) { /*ошибка*/}
        .....
        handler(ip_addr_udp,buf);
    }
}
```

Протокол POP3 (RFC-1939)

- Перед работой через протокол **POP3** сервер прослушивает **порт 110**. Когда клиент хочет использовать этот протокол, он должен создать TCP соединение с сервером. Когда соединение установлено, сервер отправляет приглашение. Затем клиент и POP3 сервер обмениваются информацией пока соединение не будет закрыто или прервано.
- **Команды POP3 состоят из ключевых слов**, за некоторыми следует один или более аргументов. Все команды заканчиваются парой CRLF. Ключевые слова и аргументы состоят из печатаемых ASCII символов. Ключевое слово и аргументы разделены одиночным пробелом. **Ключевое слово состоит от 3-х до 4-х символов**, а аргумент может быть длиной до 40-ка символов.
- Ответы в POP3 состоят из индикатора состояния и ключевого слова, за которым может следовать дополнительная информация. Ответ заканчивается парой CRLF. Существует только два индикатора состояния: **" +OK "** - положительный и **" -ERR "** - отрицательный.
- Ответы на некоторые команды могут состоять из нескольких строк. В этих случаях каждая строка разделена парой CRLF, а конец ответа заканчивается ASCII символом 46 (".") и парой CRLF.
- POP3 сессия состоит из нескольких режимов. Как только соединение с сервером было установлено и сервер отправил приглашение, то сессия переходит в режим **AUTHORIZATION (Авторизация)**. После успешной идентификации сессия переходит в режим **TRANSACTION (Передача)**. В этом режиме клиент запрашивает сервер выполнить определённые команды. Когда клиент отправляет команду QUIT, сессия переходит в режим **UPDATE**. В этом режиме POP3 сервер освобождает все занятые ресурсы и завершает работу. После этого TCP соединение закрывается.

Протокол POP3. Авторизация

Примеры

- C: USER Bob
- S: -ERR sorry, no mailbox for frated here

ИЛИ

- C: USER Bob
- S: +OK Bob is a real hoopy frood..
- C: PASS mypass
- S: +OK Bob's maildrop has 3 messages (350 octets)

ИЛИ

- C: PASS mymail
- S: -ERR maildrop already locked
- C: QUIT: +OK dewey POP3 server signing off

Протокол POP3. Транзакция

Примеры

C: STAT

S: +OK 2 350

Описание: В ответ на вызов команды сервер выдаёт положительный ответ "+OK", за которым следует количество сообщений в почтовом ящике и их общий размер в символах. Сообщения, которые помечены для удаления, не учитываются в ответе сервера.

C: LIST

S: +OK 2 messages (350 octets)

S: 1 120

S: 2 230

C: LIST 2

S: +OK 2 230

C: LIST 3

S: -ERR no such message, only 2 messages in maildrop

Описание: Если был передан аргумент, то сервер выдаёт информацию об указанном сообщении. Если аргумент не был передан, то сервер выдаёт информацию о всех сообщениях, находящихся в почтовом ящике. Сообщения, помеченные для удаления не перечисляются.

C: RETR 1

S: +OK 120 octets

S:

S: .

Описание: После положительного ответа сервер передаёт содержание сообщения

Протокол POP3. Транзакция

Примеры

C: **DELE** 1

S: +OK message 1 deleted...

C: **DELE** 2

S: -ERR message 2 already deleted

Описание: POP3 сервер помечает указанное сообщение как удалённое, но не удаляет его, пока сессия не перейдёт в режим UPDATE

C: **NOOP**

S: +OK

Описание: POP3 сервер ничего не делает и всегда отвечает положительно

C: **RSET**

S: +OK maildrop has 2 messages (350 octets)

Описание: Если какие - то сообщения были помечены для удаления, то с них снимается эта метка

Обновление

Когда клиент передаёт команду **QUIT** в режиме TRANSACTION, то сессия переходит в режим UPDATE. В этом режиме сервер удаляет все сообщения, помеченные для удаления. После этого TCP соединение закрывается.

Протокол POP3.

Дополнительные POP3 команды

Примеры:

C: **TOP** 1 10

S: +OK

S: <здесь POP3 сервер передаёт заголовки первого сообщения и первые 10-ть строк из тела сообщения.>

S:

C: **TOP** 3 100

S: -ERR no such message

Описание. Команда: **TOP** [сообщение] [n] Аргументы: [сообщение] - номер сообщения [n] - положительное число (обязательный аргумент). Если ответ сервера положительный, то после него он передаёт заголовки сообщения и указанное количество строк из тела сообщения. Возможные ответы: +OK top of message follows
-ERR no such message.

Протокол POP3. Пример сессии

S: <создаём новое TCP соединение с POP3 сервером через порт 110>

S: +OK POP3 server ready

C: **USER** Bob

S: +OK User Bob is exists

C: **PASS** mymail

S: +OK Bob's maildrop has 2 messages (350 octets)

C: **STAT**

S: +OK 2 350

C: **LIST**

S: +OK 2 messages (350 octets)

S: 1 120

S: 2 230

S: .

C: **RETR** 1

S: +OK 120 octets

S: *<После положительного ответа сервер передаёт содержание сообщения>*

S: .

C: **DELE** 1

S: +OK message 1 deleted

C: **QUIT**

S: +OK dewey POP3 server signing off (maildrop empty)

C: <закрываем соединение>

Протокол SMTP (RFC-5321)

- **Основная задача** протокола **SMTP** (Simple Mail Transfer Protocol) заключается в том, чтобы **обеспечивать передачу электронных сообщений** (почту). Для работы через протокол SMTP клиент создаёт TCP соединение с сервером через **порт 25**. Затем клиент и SMTP сервер обмениваются информацией пока соединение не будет закрыто или прервано. **Основной процедурой в SMTP является передача почты.** Далее идут процедуры перенаправления почты, проверка имён почтового ящика и вывод списков почтовых групп. Самой первой процедурой является открытие канала передачи, а последней - его закрытие.
- **Команды SMTP** указывают серверу, какую операцию хочет произвести клиент. Команды состоят из ключевых слов, за которыми следует один или более параметров. Ключевое слово состоит **из 4-х символов** и разделено от аргумента одним или несколькими пробелами. Каждая командная строка заканчивается символами **CRLF**.

Протокол SMTP. Пример сессии

- **Отправка почты**
- Первым делом **подключаемся к SMTP серверу через порт 25**. Теперь надо передать серверу команду HELO и наш IP адрес:
- C: HELO 195.161.101.33
- S: 250 smtp.mail.ru is ready *При отправке почты передаём некоторые нужные данные (отправитель, получатель и само письмо):*
- C: MAIL FROM:<frol> *указываем отправителя*
- S: 250 OK
- C: RCPT TO:<bob@mail.ru> *указываем получателя*
- S: 250 OK *указываем серверу, что будем передавать содержание письма (заголовок и тело письма)*
- C: DATA
- S: 354 Start mail input; end with <CRLF>.<CRLF> *передачу письма необходимо завершить символами CRLF.CRLF*
- S: 250 OK
- C: From: Frol frol@mail.ru
- C: To: Bob bob@mail.ru
- C: Subject: Hello *между заголовком письма и его текстом не одна пара CRLF, а две.*
- C: Hello Bob!
- C: You will be happy on next week! *заканчиваем передачу символами CRLF.CRLF*
- S: 250 OK *Теперь завершаем работу, отправляем команду QUIT:*
- S: QUIT
- C: 221 smtp.mail.ru is closing transmission channel

Протокол SMTP

Другие команды

- **SEND** - используется вместо команды MAIL и указывает, что почта должна быть доставлена на терминал пользователя.
- **SOML, SAML** - комбинации команд SEND или MAIL, SEND и MAIL соответственно.
- **RSET** - указывает серверу прервать выполнение текущего процесса. Все сохранённые данные (отправитель, получатель и др.) удаляются. Сервер должен отправить положительный ответ.
- **VRFY** - просит сервер проверить, является ли переданный аргумент именем пользователя. В случае успеха сервер возвращает полное имя пользователя.
- **EXPN** - просит сервер подтвердить, что переданный аргумент - это список почтовой группы, и если так, то сервер выводит членов этой группы.
- **HELP** - запрашивает у сервера полезную помощь о переданной в качестве аргумента команде.
- **NOOP** - на вызов этой команды сервер должен положительно ответить. NOOP ничего не делает и никак не влияет на указанные до этого данные.

Протокол TFTP

Простейший протокол пересылки файлов (**Trivial File Transfer Protocol — TFTP**) используется как очень полезное средство копирования файлов между компьютерами. TFTP передает данные **в датаграммах UDP** (при реализации в другом стеке протоколов TFTP должен запускаться поверх службы пакетной доставки данных). Для этого не потребуется слишком сложное программное обеспечение — достаточно только IP и UDP. Особенно полезен TFTP для инициализации сетевых устройств (маршрутизаторов, мостов или концентраторов). Протокол FTP определен в RFC 959, а TFTP - в RFC 1350.

Сеанс TFTP начинается запросами ***Read Request*** (запрос чтения) или ***Write Request*** (запрос записи). Клиент TFTP начинает работу после получения порта, посылая ***Read Request*** или ***Write Request*** на **порт 69** сервера.

Один из партнеров по TFTP пересылает нумерованные блоки данных одинакового размера, другой партнер подтверждает их прибытие сигналом ACK. Отправитель ожидает ACK для посланного блока до того, как пошлет следующий блок.

Характеристики TFTP

- Пересылка блоков данных размером в 512 октетов (за исключением последнего блока).
- Указание для каждого блока простого 4-октетного заголовка.
- Нумерация блоков от 1.
- Поддержка пересылки двоичных и ASCII октетов.
- Возможность чтения и записи удаленных файлов.
- Отсутствие ограничений по аутентификации пользователей.

Протокол TFTP (RFC-1350)

Каждый блок (за исключением последнего) должен иметь размер в **512 октетов** данных и завершаться **EOF (коней файла)**. Если длина файла кратна 512, то заключительный блок содержит только заголовок и не имеет никаких данных. Блоки данных нумеруются от единицы. **Каждый АСК содержит номер блока данных**, получение которого он подтверждает.

Элементы данных протокола TFTP

- Read Request (RRQ, запрос чтения)
- Write Request (WRQ, запрос записи)
- Data (DATA, данные)
- Acknowledgment (АСК, подтверждение)
- Error (ERROR, ошибка)
- Сообщение об ошибке указывает на события, подобные таким: "файл не найден" или "для записи файла на диске нет места".
- Каждый заголовок TFTP начинается операционным кодом, идентифицирующим тип элемента данных протокола (Protocol Data Unit — PDU). Форматы PDU показаны на рисунке следующего слайда.
- Отметим, что длина Read Request и Write Request меняется в зависимости от длины имени файла и полей режима, каждое из которых представляет собой текстовую строку ASCII, завершенную нулевым байтом. В поле режима могут присутствовать netascii (сетевой ASCII) или octet (октет).

Форматы элементов данных протокола TFTP

Read Request

2 байта	Строка	1 байт	Строка	1 байт
Операционный код = 1	Имя Файла	0	Режим	0

Write Request

2 байта	Строка	1 байт	Строка	1 байт
Операционный код = 2	Имя Файла	0	Режим	0

Data

2 байта	2 байта	
Операционный код = 3	Номер блока	Данные

Acknowledgment

2 байта	2 байта
Операционный код = 4	Номер блока

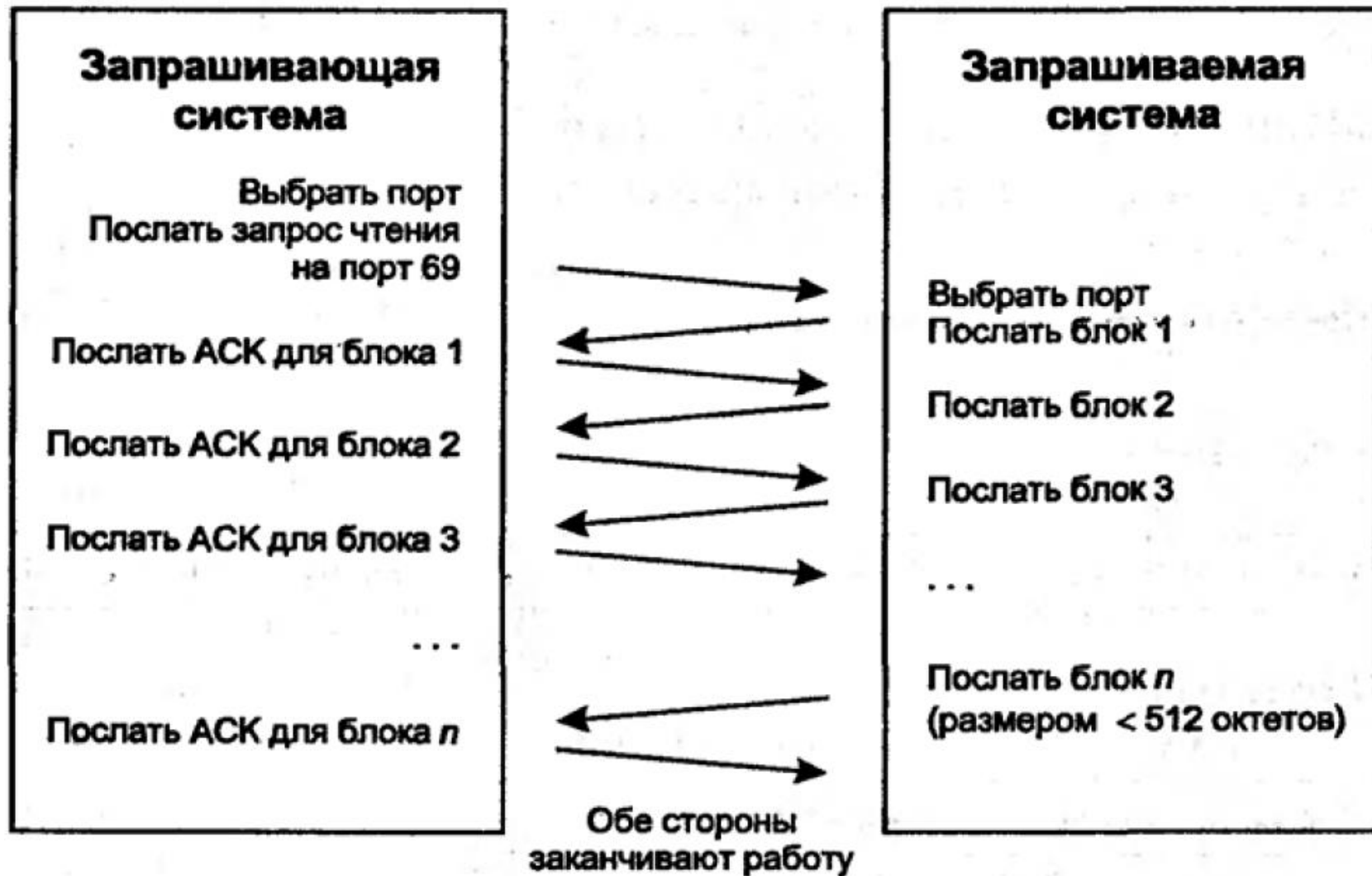
Error

2 байта	1 байт	Строка	1 байт
Операционный код = 5	Код ошибки	Сообщение об ошибке	0

Протокол TFTR

Сценарий работы

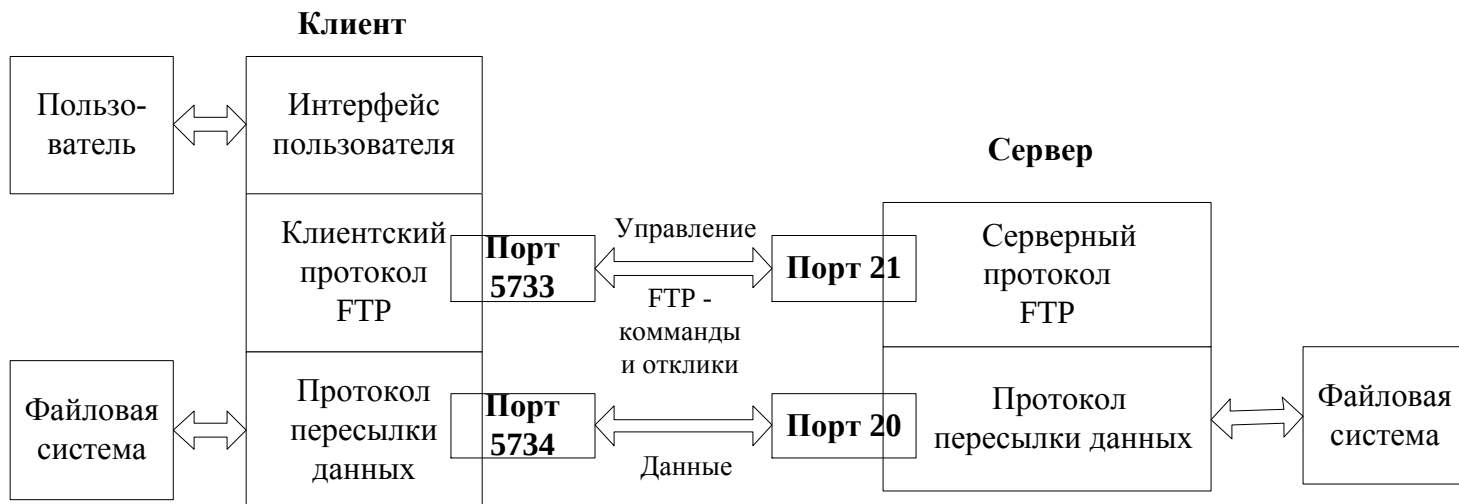
После отправки запрашиваемой стороной блока данных она переходит в режим ожидания АСК на посланный блок и, только получив этот АСК, посылает следующий блок данных.



Протокол FTP

Протокол пересылки файлов (**File Transfer Protocol – FTP**, RFC-959) реализует обмен файлами между удаленными пользователями, поверх протокола транспортного уровня **TCP**.

- **Общий алгоритм работы FTP** на пользовательском уровне содержит несколько этапов:
 - Авторизация (ввод имени-идентификатора и пароля).
 - Выбор каталога.
 - Определение режима обмена (поблочный, поточный, ASCII или двоичный).
 - Выполнение команд обмена (get, ls или dir, del, put или send).
 - Завершение процедуры (quit или close).
- **Спецификация FTP определяет общедоступный или частный доступ к файлам:**
 - доступ к общедоступным файлам через анонимную регистрацию (логин – ANONYMOUS);
 - доступ к личным файлам, разрешенный только для определенных пользователей (требуется логин и пароль в режиме авторизации).
 - Протокол FTP использует **два порта**: один порт используется для **передачи команд** (например, 21 порт), второй порт используется для **передачи данных**.



Протокол FTP

Команды пересылки файлов позволяют:

- копировать одиночный файл между хостами;
- копировать несколько файлов между хостами;
- добавлять содержимое локального файла к удаленному файлу;
- копировать файл и добавлять к его имени номер для формирования уникального имени (например, файлы ежедневной регистрации получают имена log. 1, log.2 и т.д.).

Команды обслуживания файлов разрешают:

- просмотреть список файлов каталога;
- узнать текущий каталог и изменить его на другой;
- создавать и удалять каталоги;
- переименовывать или удалять файлы.

Управляющие команды служат для:

- идентификации пересылки файлов ASCII, EBCDIC или двоичных файлов;
- проверки структурирования файла (как последовательность байт или как последовательность записей);
- указания способа пересылки файла (например, как поток октетов).

Протокол FTP

```
csc.sibsutis> ftp
```

```
ftp> help
```

Commands may be abbreviated. Commands are:

!	?	acct	append	ascii	binary	bye	cd	debug
delete	dir	drive	exit	fcd	fdir	fpwd	get	help
iget	image	iput	lcd	ldir	lmkdir	local	login	lpwd
ls	mdelete	mget	mkdir	mput	option	parent	passive	put
pwd	quit	quote	rename	retrieve	rmdir	send	server	show
stat	store	take	tenex	tget	tput	type	user	verbose

Протокол FTP

Стоит отметить, что всякий раз, как только выполняется вызов на просмотр списка директории, копирование файла и т.д., то для пересылки этой информации открывается и используется соединение для данных. Процесс открытия этого соединения зависит от выбранного режима: *активный* или *пассивный*. Выбор режима определяется клиентом. В случае активного режима, клиент посылает команду – **PORT** <IP адрес, порт>, после чего серверная сторона иницирует соединение на указанную оконечную точку. В случае пассивного режима, клиент посылает команду **PASV**, ждет от сервера <IP адрес, порт> для инициации соединения на указанный адрес. После пересылки данных соединение по второму порту закрывается, а общение клиента с сервером продолжается по первому порту.

Пример пересылки файлов в активном режиме

```
csc-sibsutis>ftp
```

```
ftp>open sibsutis.ru
```

```
Connected to sibsutis.ru.
```

```
220 sibsutis.ru FTP server ready.
```

Далее проводим авторизацию (USER, PASS).

```
ftp>dir
```

```
-->PORT 128,36,4,22,10,53
```

Здесь IP-адрес 128.36.4.22 будет записан как 128,36,4,22, а порт 2613 — как 10,53

```
200 PORT command successful.
```

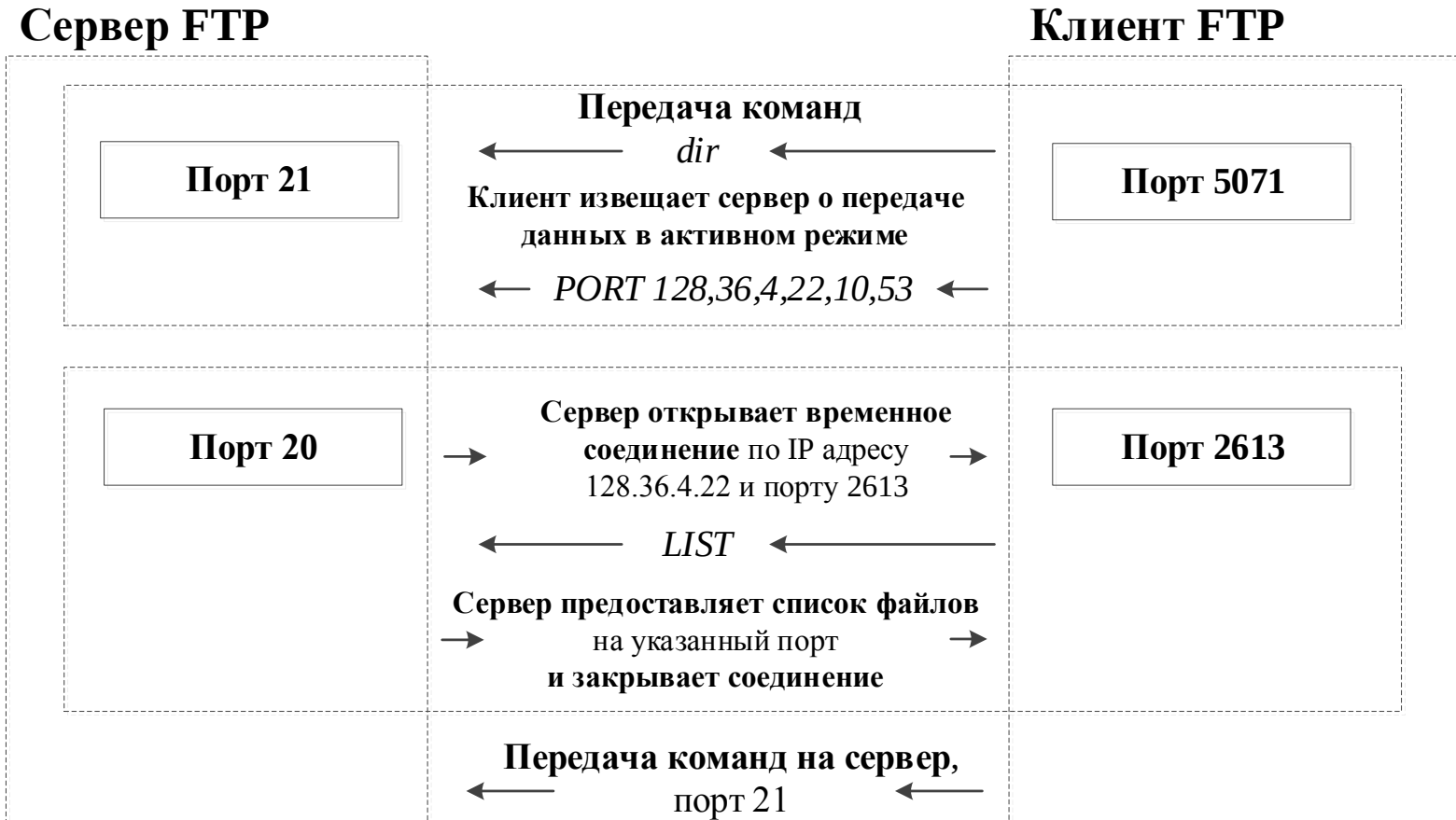
```
—> LIST
```

Далее сервер открывает соединение с объявленным клиентом портом и предоставляет список файлов.

*Сразу после пересылки списка файлов соединение данных будет закрыто. Затем, аналогично, мы можем получить , например, необходимый файл через команду **get** (> get article).*

Протокол FTP

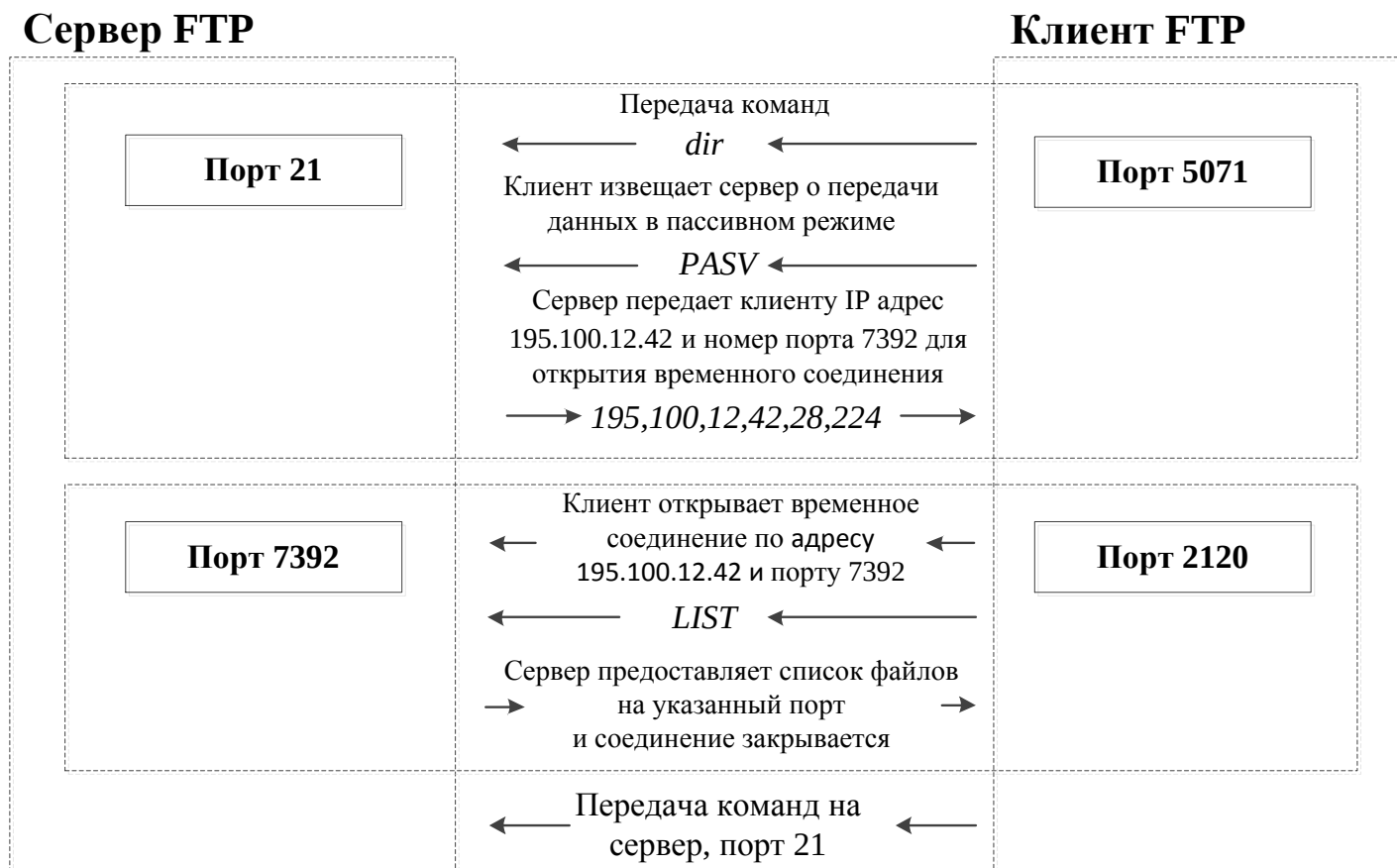
Пример пересылки файлов в активном режиме



Протокол FTP

Пассивный режим

- Как было сказано, существует альтернативный сценарий. Если клиент посылает команду *PASV*, сервер возвращает IPv4 адрес и номер порта (в числовой записи, аналогичной активному соединению) и переходит к прослушиванию установки соединения данных от клиента.
- Ранее преобладало использование команды *PORT*. Однако теперь клиент может послать команду *PASV* для пересылки файлов через простую систему защиты (firewall), которая не разрешает установку соединений из поступающих сообщений.



Протокол FTP

Типы данных, структуры файлов и методы пересылки

- На обоих концах соединения необходимо обеспечить единый формат для пересылаемых данных. Этот файл текстовый или двоичный? Он структурирован по записям или по блокам?
- Для описания формата пересылки используются три атрибута: *тип данных* (data type), *структура файла* (file structure) и *режим пересылки* (transmission mode). Допустимые значения этих атрибутов рассмотрены ниже. В общем случае применяются:
 - пересылка текста ASCII или двоичных данных;
 - неструктурированный файл, который рассматривается как последовательность байт;
 - режим пересылки рассматривает файл как поток байт.

Протокол FTP

Типы данных

- Файл может содержать текст ASCII, EBCDIC или двоичный образ данных (существует еще тип, называемый *локальным* или *логическим байтом* и применяемый для компьютеров с размером байта в 11 бит). Текстовый файл может содержать обычный текст или текст, форматированный для вывода на принтер.
- Типом данных по умолчанию является не распечатываемый текст ASCII (текст без управляющих символов форматирования). Тип данных может быть изменен стандартной командой *TYPE*, пересылаемой по управляющему соединению.

Пересылка текста ASCII

- Хотя текст ASCII является стандартным, компьютеры интерпретируют его по-разному из-за различия в кодах конца строки. Системы Unix используют для этого <LF>, компьютеры PC — <CR><LF>, а Macintosh — <CR>.

Пересылка текста EBCDIC

- Поддерживающие кодировку EBCDIC хосты обеспечивают весьма полезную команду пользовательского интерфейса, иницирующую пересылку по управляющему соединению команды *TYPE E*.

Пересылка двоичных данных

- С пересылки текстов ASCII легко переключиться на двоичный образ данных. В текстовом пользовательском интерфейсе для этого служит команда *binary*, а в графическом — командная кнопка *binary* (двоичные данные). Клиент меняет тип пересылаемых данных командой *TYPE I*, передаваемой по управляющему соединению.

Протокол FTP

Структуры файлов

- В FTP поддерживаются две структуры:
- *файловая структура*, соответствующая неструктурированному файлу, который рассматривается как последовательность байт;
- *структура записей*, которая применяется для файлов, состоящих из последовательности записей.
- Более распространена *файловая структура*, которая применяется по умолчанию. Перейти на *структуру записей* можно стандартной командой *STRU R*, пересылаемой по управляющему соединению.

Протокол FTP

Режимы пересылки

- Режим пересылки и структура файла определяют, как будут форматированы данные для обмена по соединению. Существуют три режима пересылки: *stream* (поток), *block* (блочный режим) и *compressed* (сжатые данные).
- В режиме потока и файловой структуры файл передается как поток байт. FTP возлагает на ТСПР обеспечение целостности данных и не включает в данные никаких заголовков или разделителей. Единственным способом указания на конец файла будет нормальное завершение соединения для данных.
- Для режима потока и структуры записей каждая запись отделяется 2-байтовым управляющим кодом конца записи (End Of Record — EOR), а конец файла отмечается символами конца файла (End Of File — EOF). EOR кодируется как X'FF 01, а EOF — X'FF02. Для последней записи файла EOR и EOF записываются как X'FF 03. Если файл содержит байт данных из одних единиц, то такой байт представляется при пересылке как X'FFFF.
- В блочном режиме файл пересылается как последовательность блоков данных. Каждый блок начинается 3-байтовым заголовком
- Режим сжатия данных используется крайне редко, поскольку обеспечивает очень неудачный метод архивирования, разрушающий последовательность повторяющихся байт. Обычно пользователю проще применить одну из более удачных программ сжатия, широкодоступных на современных компьютерах, и далее пересылать полученный архивный файл как двоичные данные.

Режим потока наиболее распространен и используется по умолчанию. Изменить его на блочный режим можно стандартной командой *MODE B*, пересылаемой по управляющему соединению.

Преимущество структуры записей или блочного режима, состоит в том, что будет явно отмечен конец файла и после завершения его пересылки можно сохранить соединение для данных, а, следовательно, использовать его для нескольких пересылок.

СПАСИБО ЗА ВНИМАНИЕ